



CSE387 – Data Mining and Business Intelligence

Project Title: Mushroom Classification

Submitted to:
Dr. Mahmoud Mounir
Eng. Esraa Karam
Submitted by:

Rawan Essam Hussein	23P0309
Mai Hamed Hussein Ahmed	23P0261
Maryam Hamdy Hassan	23p0260
Mariam Riyad Freeg Aboelyazed	23p0147
Rawan Hany Shaker Hussein	23P0393
Basmala Hany Mohamed Taha	23P0008

Date of Submission:
6 December, 2025

Table of Contents

Phase 1	5
1.1. Project Overview	5
Objective	5
Dataset	5
Goal	5
1.2. Dataset Understanding	5
Loading & Shape	5
Target Variable:	5
Missing Values:	5
Duplicates:	5
Unique Categories:	5
Understanding the Dataset Features (Attributes)	5
Code:	6
Output:	7
1.3. Data Preprocessing	9
Missing Value Handling	9
Data Type Validation	9
Optional Encoding	9
Normalization/Scaling	9
Code:	10
Output:	11
1.4. Outlier Detection	12
4.1 Frequency-Based Outliers	12
Code:	13
Figures:	15
4.2 Mode-Based Outliers	21
Code:	22
4.3 K-Modes Clustering Outliers	29
Code:	30
Output:	31
4.4 Comments on Outliers (Why They Were Kept)	33
1.5. Correlation Analysis	33

5.1 Cramér's V (Symmetric Association).....	33
Code:	34
Observations:	34
5.2 Theil's U (Asymmetric Directional Dependency)	35
Code:	36
Observations:	36
5.3 Visual Confirmation.....	37
Code:	38
Observations:	38
Observations:	39
5.4 Comparison and Interpretation	39
Cramér's V Heatmap (0–1):	39
Theil's U (Uncertainty Coefficient):	40
Interpretation:	40
Pearson's r Removal:	40
1.6. Tools and Libraries Used	40
1.7. Summary of Findings.....	41
Phase 2:	42
2. Classification Modeling and Evaluation	42
2.1. Data Preparation for Classification	42
2.2. Decision Tree Classifier.....	43
Model Training	43
Model Performance	48
Classification Report for Decision Tree Classifier	48
Confusion Matrix.....	48
Feature Importance	49
Decision Tree Visualization	49
2.3. K-Nearest Neighbors (KNN)	51
Code	51
Model Performance	52
Confusion Matrix	53
Interpretation	53
2.4. Random Forest Classifier	54

Code	54
Model Performance	56
Confusion Matrix.....	56
Feature Importance	56
2.5. Naïve Bayes Classifier	58
Code	58
Model Performance	59
Confusion Matrix.....	59
2.6. Model Comparison	60
Code	60
2.6.1. Loaded Models and Data Consistency	65
Best Overall Model: Decision Tree.....	70
Additional Notes:	70
2.7. Tools and Libraries Used in Phase 2	71

Table of Figures

Figure 1 – Class Distribution	9
Figure 2 – Category Frequencies (red = rare, blue = common)	15
Figure 3 – Outlier Count per Feature (horizontal bar showing how many rare values per attribute)	21
Figure 4– Outlier Flag Heatmap	21
Figure 5– Mode Analysis: Per-feature frequency bars with a red dashed line marking the 5 % threshold used to classify rare categories.	23
Figure 6 – Cluster Sizes (Excluding Outliers)	32
Figure 7 – Outliers per Cluster (Rare Categories)	32
Figure 8 – Cramér’s V Heatmap	35
Figure 9 – Theil’s U Heatmap $U(X Y)$	37
Figure 11 – Jitter Scatter (Strong vs Weak Pairs).....	39
Figure 12 – Decision Tree Confusion Matrix	48
Figure 13 – Top 10 Feature Importances (Decision Tree)	49
Figure 14 – Simplified Decision Tree (depth = 3)	50
Figure 15 – KNN Confusion Matrix	53
Figure 16 – Random Forest Confusion Matrix	56
Figure 17 – Top 10 Feature Importances (Random Forest)	57
Figure 18 – Naïve Bayes Confusion Matrix	59
Figure 19- All Model Performance Comparison	66
Figure 20- Confusion Matrices All Models	67

Figure 21 – ROC Curves for Models with Probability Outputs.....	68
Figure 22 – AUC Score Comparison	69

Phase 1

1.1. Project Overview

Objective

Analyze a categorical mushroom dataset to detect and interpret outliers, study associations between features, and prepare the data for accurate classification in the next phase.

Dataset Source

Mushroom Dataset (mushrooms.csv) containing morphological features such as cap shape, color, odor, and gill size, each labeled as edible (e) or poisonous (p).

Goal of Phase 1

Perform preprocessing, outlier detection using categorical methods, and visualize correlations between nominal variables. *(Classification models will follow in Phase 2)*

1.2. Dataset Understanding

Loading & Shape: Loaded using pandas → 8124 rows × 23 columns.

Target Variable: class — e (edible) and p (poisonous).

Missing Values: Handled in preprocessing.

Duplicates: Checked — none found.

Unique Categories: Listed per column to understand nominal structure.

Understanding the Dataset Features (Attributes)

Each mushroom record contains the following descriptive categorical features:

- 1) **cap-shape, cap-surface, cap-color** – physical properties of the mushroom cap.
- 2) **bruises, odor** – indicators of possible toxicity.
- 3) **gill-attachment, gill-spacing, gill-size, gill-color** – structure and color of gills.
- 4) **stalk-shape, stalk-root, stalk-surface - above/below - ring, stalk - color-above/below - ring** – features of the stalk.

- 5) **veil-type, veil-color** – veil morphology.
- 6) **ring-number, ring-type** – presence and type of ring.
- 7) **spore-print-color, population, habitat** – environmental characteristics.

These attributes are all nominal and collectively determine whether a mushroom is edible or poisonous.

Code:

```
from google.colab import files
uploaded = files.upload()
```

Choose Files mushrooms.csv
mushrooms.csv(text/csv) - 374003 bytes, last modified: 10/24/2025 - 100% done
Saving mushrooms.csv to mushrooms.csv

```
# Import dataset using pandas
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler

na_values = '?'
data = pd.read_csv('mushrooms.csv', na_values='?')

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
print("Full Dataset:\n")
print(data)

# No. of rows & columns
print("\nShape of Dataset:", data.shape)

# Column name
print("\nColumn Names:\n", data.columns.tolist())

# Type of data in each column
print("\nData Types:\n", data.dtypes)

# No. of duplicate rows
duplicates = data.duplicated().sum()
print("\nNumber of duplicate rows:", duplicates)

# Missing values per column
missing = data.isnull().sum()
print("\nMissing values per column:\n", missing)
```

```

# Unique values
for column in data.columns:
    print(f"\nUnique values in '{column}':")
    print(data[column].unique())

print("\nCategory counts per feature:")
for col in data.columns:
    print(f"{col}: {data[col].nunique()} unique categories")

#Plot distribution of edible vs poisonous mushrooms

data['class'].value_counts().plot(kind='bar', color=['green', 'red'])
plt.title('Distribution of Edible vs Poisonous Mushrooms')
plt.xlabel('Class')
plt.ylabel('Count')
plt.show()

# summary of dataset characteristics

summary = pd.DataFrame({
    'Total Rows': [data.shape[0]],
    'Total Columns': [data.shape[1]],
    'Duplicate Rows': [duplicates],
    'Missing Values': [missing.sum()],
    'Edible Count': [data['class'].value_counts().get('e', 0)],
    'Poisonous Count': [data['class'].value_counts().get('p', 0)]
})

print("\nDataset Summary Table:\n")
print(summary)

```

Output:

Streaming output truncated to the last 5000 lines.

3124	k	s	d
3125	n	y	d
3126	k	y	d
3127	n	v	d
3128	n	v	d
3129	n	y	d
3130	k	v	d
3131	n	y	d
3132	k	y	d
3133	n	v	d
3134	k	y	d
3135	k	y	d
3136	k	v	d
3137	n	v	d
3138	h	v	p
3139	n	y	d
3140	k	v	d
3141	h	y	d
3142	k	y	d

Number of duplicate rows: 0

Missing values per column:

class	0
cap-shape	0
cap-surface	0
cap-color	0
bruises	0
odor	0
gill-attachment	0
gill-spacing	0
gill-size	0
gill-color	0
stalk-shape	0
stalk-root	2480
stalk-surface-above-ring	0
stalk-surface-below-ring	0
stalk-color-above-ring	0
stalk-color-below-ring	0
veil-type	0
veil-color	0
ring-number	0
ring-type	0
spore-print-color	0
population	0
habitat	0

dtype: int64

Category counts per feature:

class: 2 unique categories
cap-shape: 6 unique categories
cap-surface: 4 unique categories
cap-color: 10 unique categories
bruises: 2 unique categories
odor: 9 unique categories
gill-attachment: 2 unique categories
gill-spacing: 2 unique categories
gill-size: 2 unique categories
gill-color: 12 unique categories
stalk-shape: 2 unique categories
stalk-root: 4 unique categories
stalk-surface-above-ring: 4 unique categories
stalk-surface-below-ring: 4 unique categories
stalk-color-above-ring: 9 unique categories
stalk-color-below-ring: 9 unique categories
veil-type: 1 unique categories
veil-color: 4 unique categories
ring-number: 3 unique categories
ring-type: 5 unique categories
spore-print-color: 9 unique categories
population: 6 unique categories
habitat: 7 unique categories

Dataset Summary Table:

	Total Rows	Total Columns	Duplicate Rows	Missing Values	Edible Count \
0	8124	23	0	2480	4208
	Poisonous Count				
0	3916				

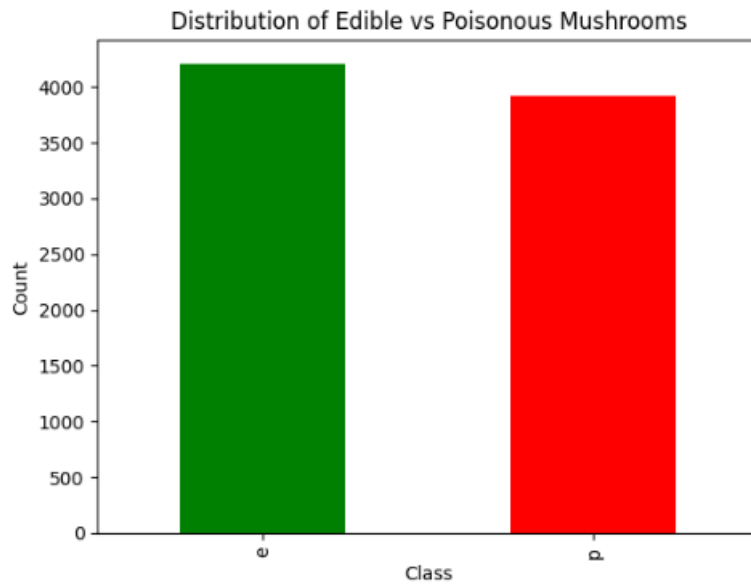


Figure 1 – Class Distribution

1.3. Data Preprocessing

Missing Value Handling

Missing entries ('?') replaced using `SimpleImputer(strategy='most_frequent')`.

Data Type Validation

All columns confirmed categorical (object).

Optional Encoding

Created `encoded_data` copy with `LabelEncoder` for future numerical models; original data remained categorical for Phase 1 analysis.

Normalization/Scaling

Not applied; not meaningful for nominal features.

Code:

```
# Preprocessing part
imp_mean = SimpleImputer(missing_values=np.nan, strategy='most_frequent')
data = pd.DataFrame(imp_mean.fit_transform(data), columns=data.columns)

print("\nData Before Label Encoding:\n")
print(data.head())

#Encode Categorical Data and saving it as a copy if needed
encoded_data = data.copy()

labelencoder = LabelEncoder()

for col in encoded_data.columns:
    encoded_data[col] = labelencoder.fit_transform(encoded_data[col])

print("\nOriginal Data (unchanged):\n")
print(data.head())

print("\nLabel Encoded Data:\n")
print(encoded_data.head())

# Check data types after label encoding
print(encoded_data.dtypes)

# Select numeric columns
numeric_cols = encoded_data.select_dtypes(include=[np.number]).columns
print("Numeric columns:", numeric_cols)
```

Output:

Data Before Label Encoding:

```
class cap-shape cap-surface cap-color bruises odor gill-attachment \
0    p      x      s      n      t      p      f
1    e      x      s      y      t      a      f
2    e      b      s      w      t      l      f
3    p      x      y      w      t      p      f
4    e      x      s      g      f      n      f

gill-spacing gill-size gill-color stalk-shape stalk-root \
0          c      n      k      e      e
1          c      b      k      e      c
2          c      b      n      e      c
3          c      n      n      e      e
4          w      b      k      t      e

stalk-surface-above-ring stalk-surface-below-ring stalk-color-above-ring \
0          s      s      w
1          s      s      w
2          s      s      w
3          s      s      w
4          s      s      w

stalk-color-below-ring veil-type veil-color ring-number ring-type \
0          w      p      w      o      p
1          w      p      w      o      p
2          w      p      w      o      p
3          w      p      w      o      p
4          w      p      w      o      e

spore-print-color population habitat
0          k      s      u
1          n      n      g
2          n      n      m
3          k      s      u
4          n      a      g
```

Label Encoded Data:

```
class cap-shape cap-surface cap-color bruises odor gill-attachment \
0    1      5      2      4      1      6      1
1    0      5      2      9      1      0      1
2    0      0      2      8      1      3      1
3    1      5      3      8      1      6      1
4    0      5      2      3      0      5      1

gill-spacing gill-size gill-color stalk-shape stalk-root \
0          0      1      4      0      2
1          0      0      4      0      1
2          0      0      5      0      1
3          0      1      5      0      2
4          1      0      4      1      2

stalk-surface-above-ring stalk-surface-below-ring stalk-color-above-ring \
0          2      2      7
1          2      2      7
2          2      2      7
3          2      2      7
4          2      2      7

stalk-color-below-ring veil-type veil-color ring-number ring-type \
0          7      0      2      1      4
1          7      0      2      1      4
2          7      0      2      1      4
3          7      0      2      1      4
4          7      0      2      1      0

spore-print-color population habitat
0          2      3      5
1          3      2      1
2          3      2      3
3          2      3      5
4          3      0      1
```

```

class                int64
cap-shape            int64
cap-surface          int64
cap-color            int64
bruises              int64
odor                 int64
gill-attachment      int64
gill-spacing         int64
gill-size            int64
gill-color           int64
stalk-shape          int64
stalk-root           int64
stalk-surface-above-ring int64
stalk-surface-below-ring int64
stalk-color-above-ring int64
stalk-color-below-ring int64
veil-type            int64
veil-color           int64
ring-number          int64
ring-type            int64
spore-print-color     int64
population            int64
habitat              int64
dtype: object
Numeric columns: Index(['class', 'cap-shape', 'cap-surface', 'cap-color', 'bruises', 'odor',
                        'gill-attachment', 'gill-spacing', 'gill-size', 'gill-color',
                        'stalk-shape', 'stalk-root', 'stalk-surface-above-ring',
                        'stalk-surface-below-ring', 'stalk-color-above-ring',
                        'stalk-color-below-ring', 'veil-type', 'veil-color', 'ring-number',
                        'ring-type', 'spore-print-color', 'population', 'habitat'],
                        dtype='object')

```

1.4. Outlier Detection

Outlier detection in this dataset focuses on **nominal (categorical)** attributes.

Since there are no numerical values, outliers were identified based on **rare category frequencies** and **cluster behavior** using three complementary methods.

All approaches are unsupervised and preserve the categorical nature of the data.

4.1 Frequency-Based Outliers

This method examines how often each category appears within a feature. Any category that occurs in less than 5 % of the dataset is flagged as rare. Such low-frequency values may represent unusual mushroom types or rare physical characteristics. This approach helps detect attributes like uncommon cap colors, stalk textures, or odors that only appear a few times in the dataset.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

nominal_data = data.select_dtypes(include=['object', 'category'])

# Frequency threshold
threshold = 0.05

outlier_rows = pd.Series(False, index=data.index)

for col in nominal_data.columns:
    frequencies = nominal_data[col].value_counts(normalize=True)

    rare_values = frequencies[frequencies < threshold].index
    outlier_rows |= nominal_data[col].isin(rare_values)

    plt.figure(figsize=(8, 4))
    plt.title(f"Frequency of Categories in '{col}' (Outliers < {threshold*100:.0f}%)")
    plt.xlabel("Category")
    plt.ylabel("Frequency (%)")

    freq_percent = frequencies * 100

    # Color bars: red = rare, blue = normal
    colors = ['red' if val in rare_values else 'blue' for val in frequencies.index]

    plt.bar(frequencies.index, freq_percent, color=colors)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

print(f"\n👉 Number of outlier rows (frequency < {threshold*100}%) : {outlier_rows.sum()}")
```

```

##### Outlier Count per Feature
outlier_counts = {}

for col in nominal_data.columns:
    freq = nominal_data[col].value_counts(normalize=True)
    rare_vals = freq[freq < threshold].index
    outlier_counts[col] = nominal_data[col].isin(rare_vals).sum()

plt.figure(figsize=(10,6))
pd.Series(outlier_counts).sort_values().plot(kind='barh', color='orange', edgecolor='black')
plt.xlabel('Number of Outlier Rows')
plt.title(f'Outlier Count per Feature (Frequency < {threshold*100:.0f}%)')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

##### Outlier Binary Heatmap (rows × features)
rare_values_per_col = {}
for col in nominal_data.columns:
    freq = nominal_data[col].value_counts(normalize=True)
    rare_values_per_col[col] = freq[freq < threshold].index

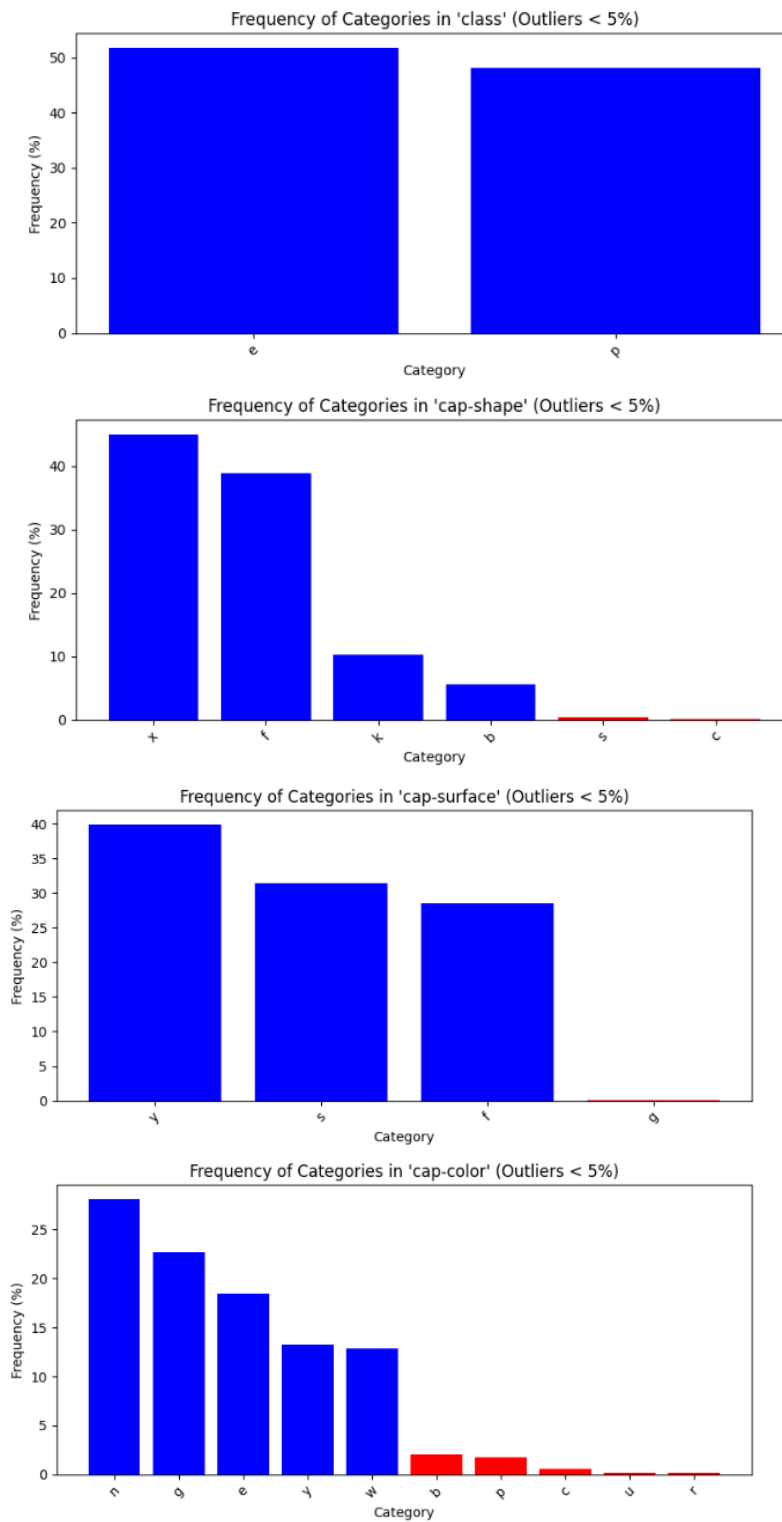
flags_df = pd.DataFrame(index=nominal_data.index)
for col in nominal_data.columns:
    flags_df[col] = nominal_data[col].isin(rare_values_per_col[col])

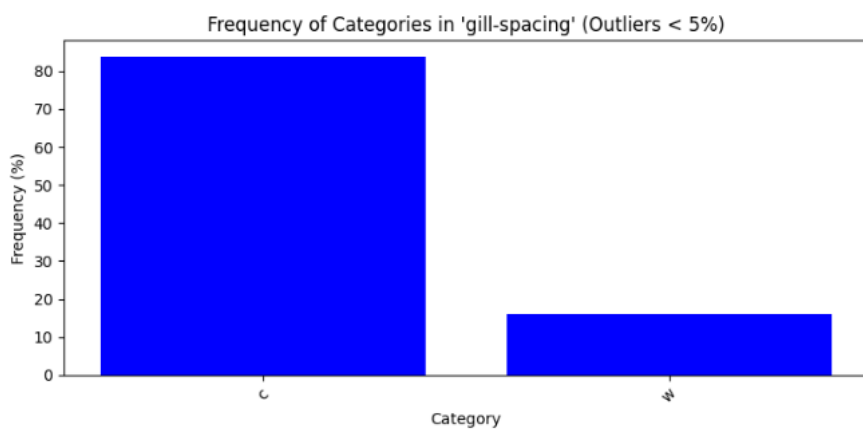
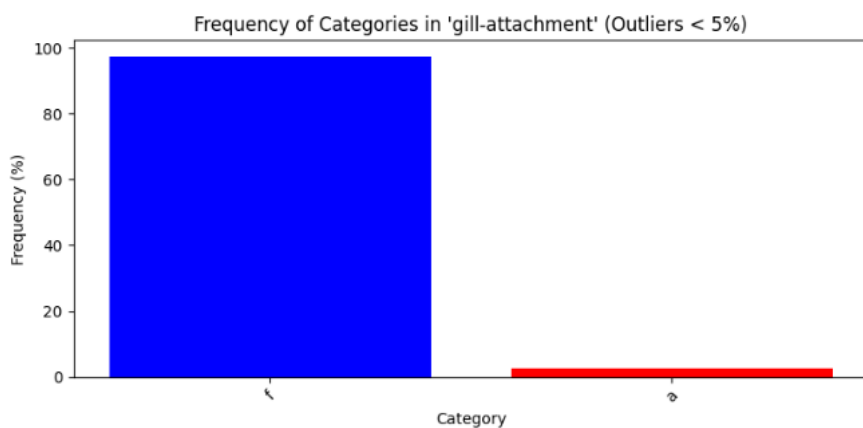
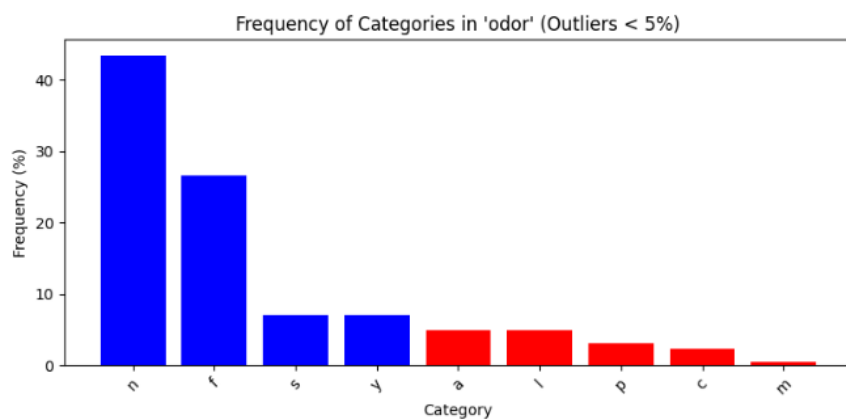
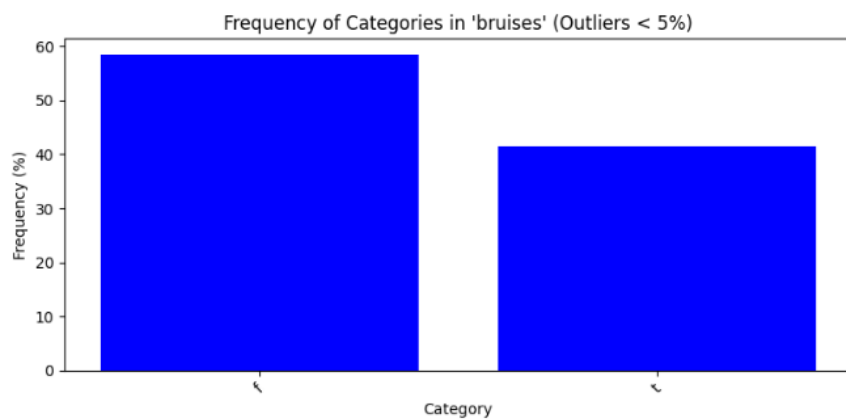
plt.figure(figsize=(12,6))
plt.imshow(flags_df.astype(int), aspect='auto', interpolation='none', cmap='coolwarm')
plt.title('Outlier Flags Heatmap (1 = Rare Category)')
plt.xlabel('Features')
plt.ylabel('Rows')
plt.colorbar(label='Flag (1=Outlier)')
plt.xticks(ticks=np.arange(len(nominal_data.columns)), labels=nominal_data.columns, rotation=90)
plt.tight_layout()
plt.show()

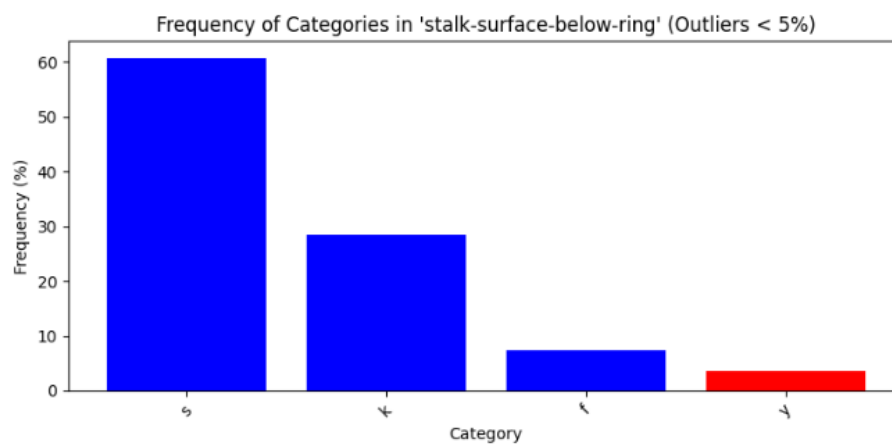
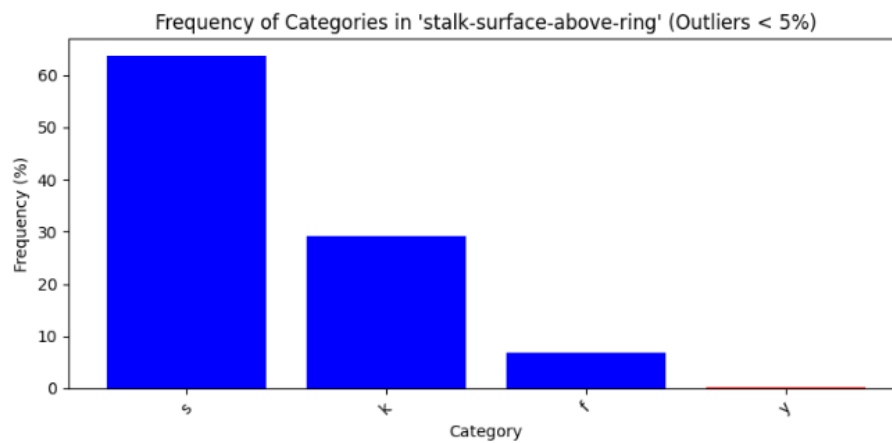
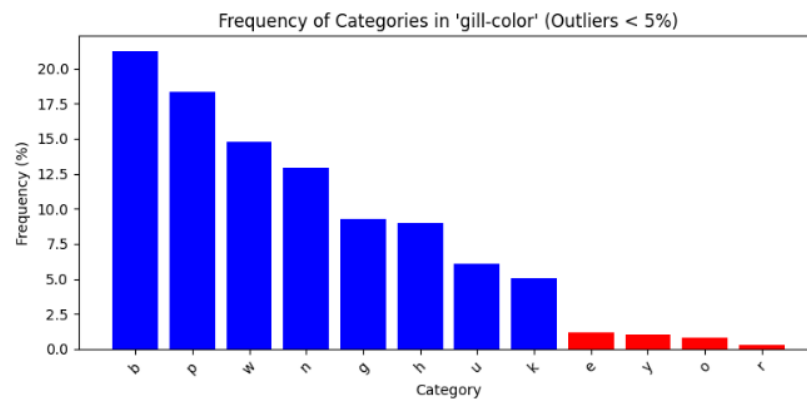
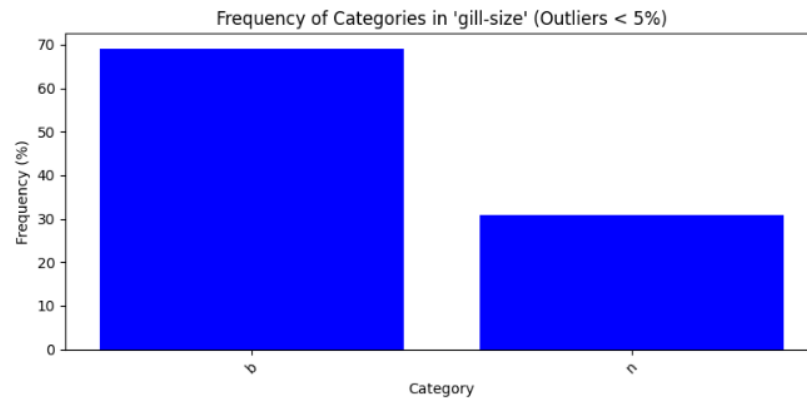
```

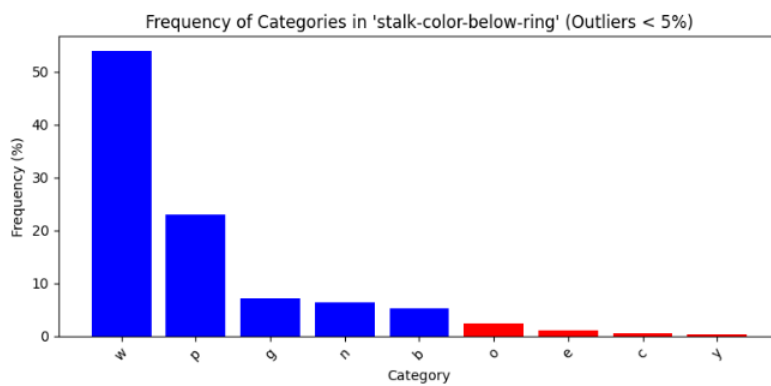
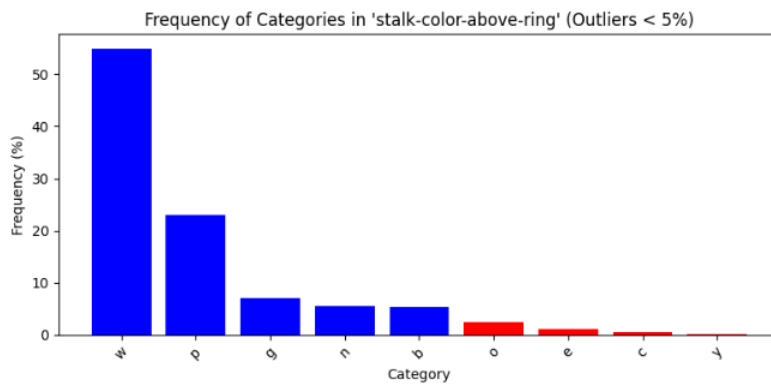
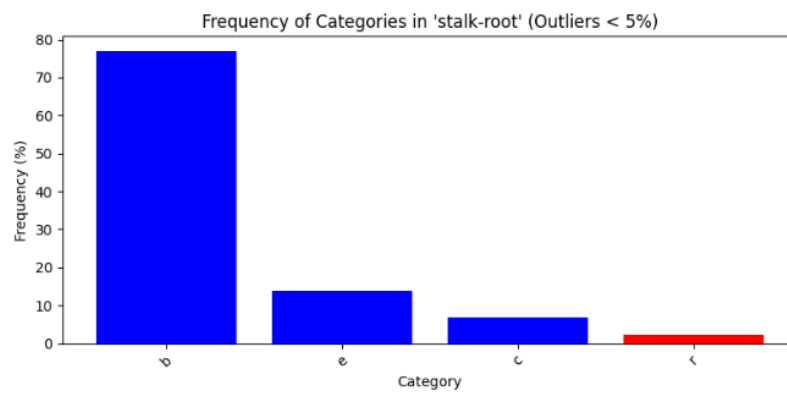
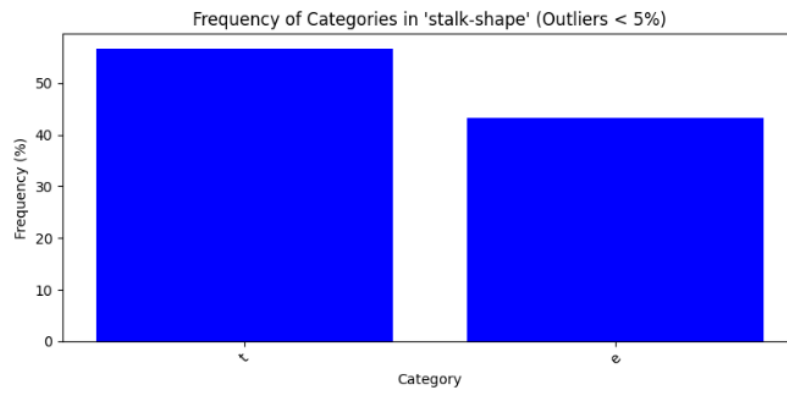
Figures:

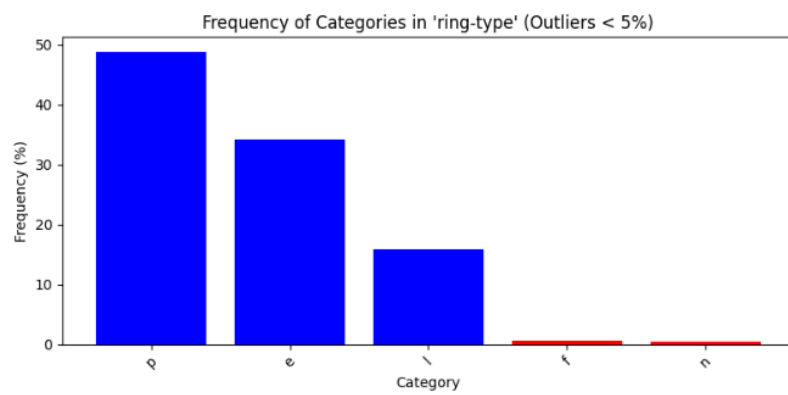
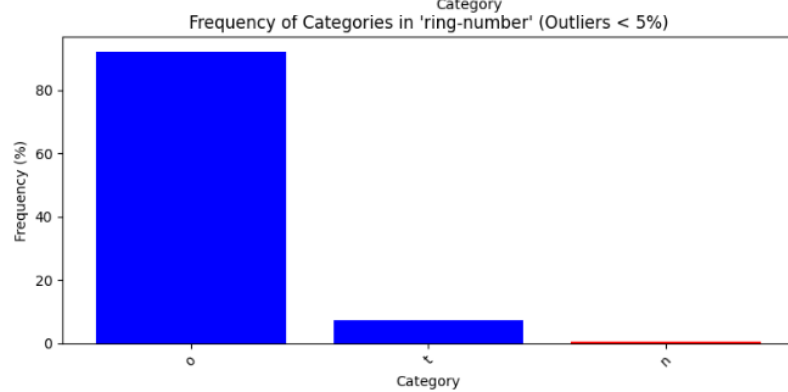
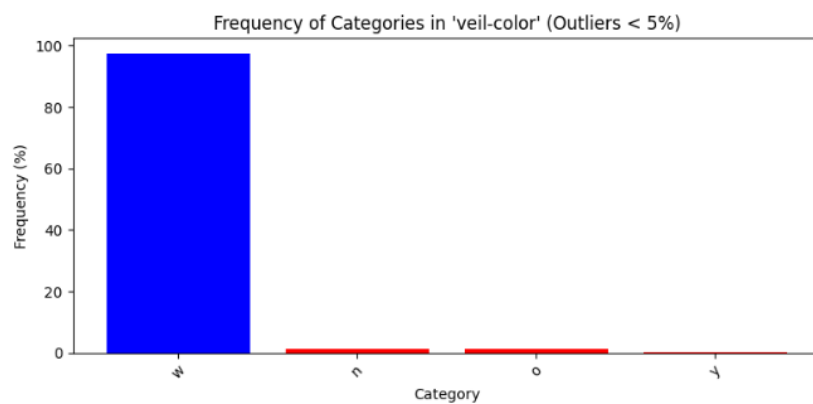
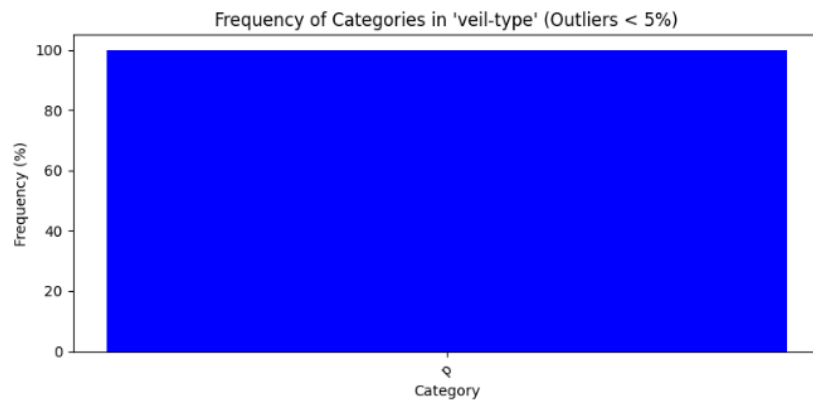
Figure 2 – Category Frequencies (red = rare, blue = common)











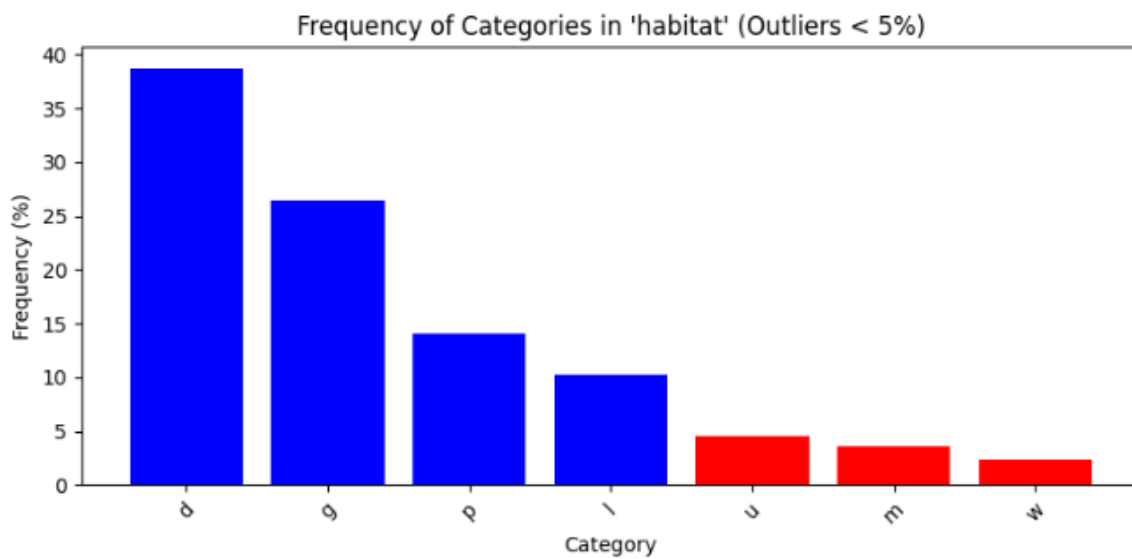
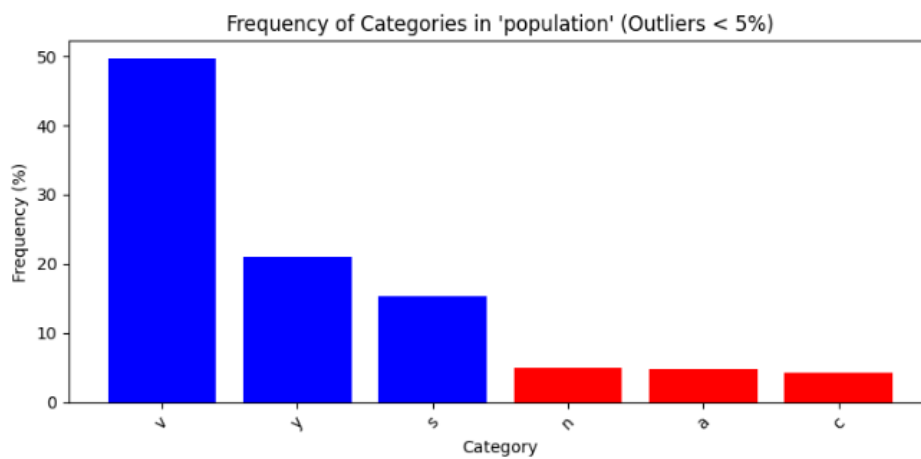
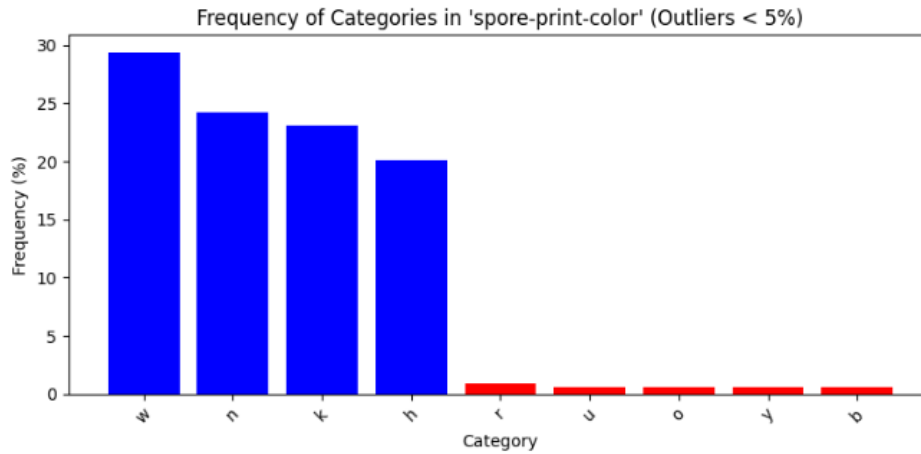


Figure 3 – Outlier Count per Feature (horizontal bar showing how many rare values per attribute)

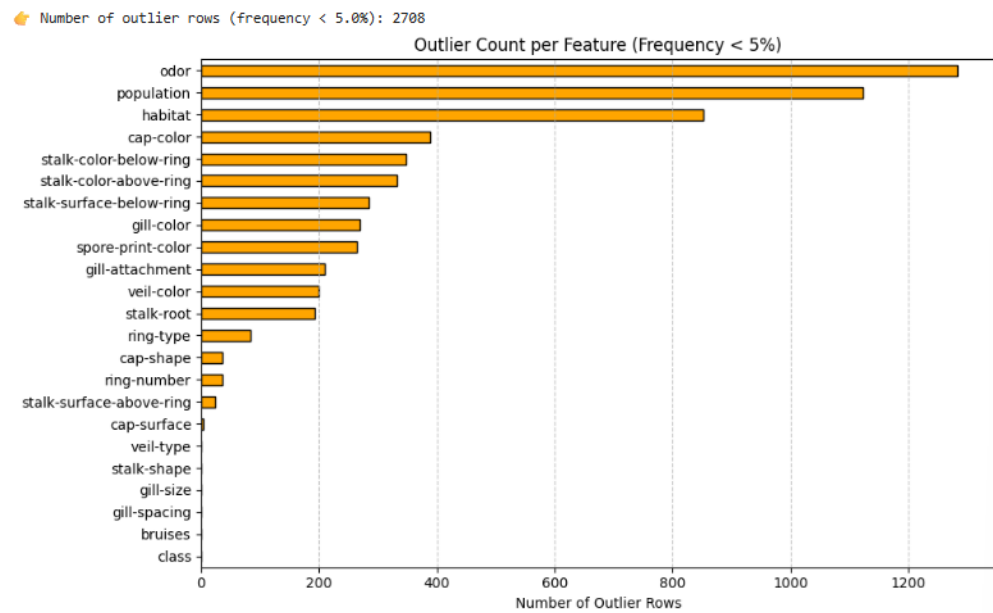
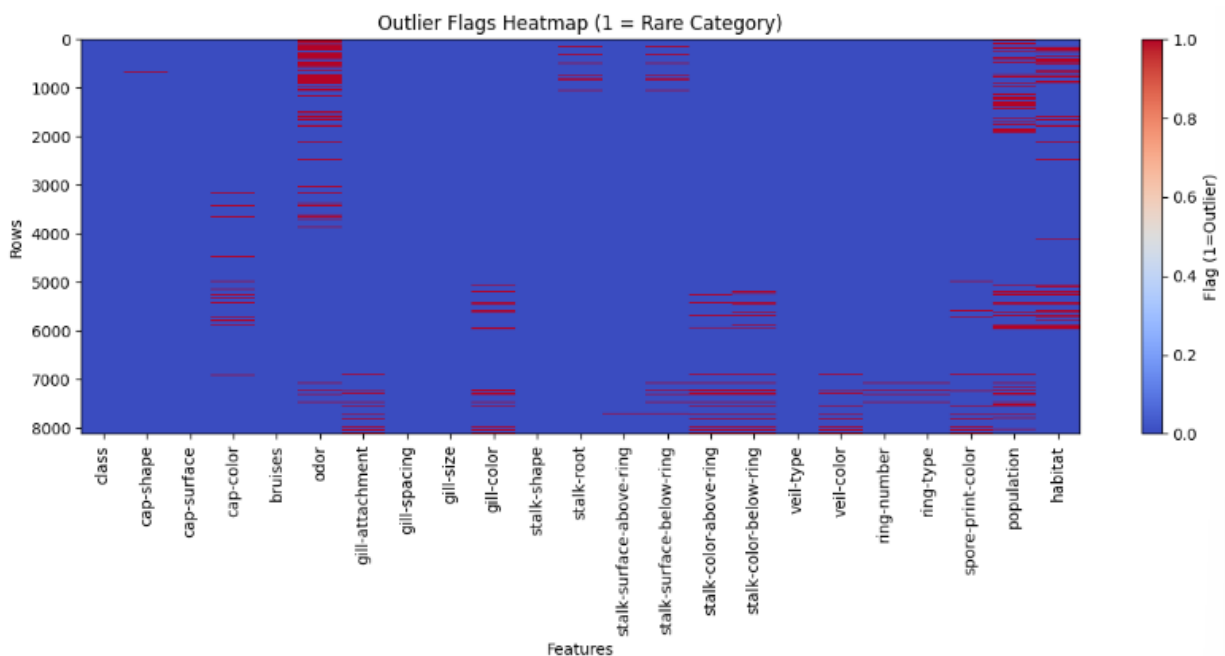


Figure 4– Outlier Flag Heatmap



4.2 Mode-Based Outliers

The mode-based approach identifies categories that deviate strongly from the most frequent (mode) value of each feature. Values occurring below 5 % of the mode frequency are labeled as potential outliers. This captures features where one dominant category exists (e.g., most mushrooms share the same gill-size), but a few records show

unusual values. It complements the frequency method by emphasizing deviation from the main trend rather than absolute rarity.

Code:

```
def mode_outlier_detection(data, threshold=0.05):
    outlier_rows = pd.Series(False, index=data.index)

    for col in data.columns:
        mode_value = data[col].mode()[0]
        mode_frequency = (data[col] == mode_value).mean()

        value_frequencies = data[col].value_counts(normalize=True)
        rare_values = value_frequencies[value_frequencies < threshold].index

        outlier_rows |= data[col].isin(rare_values)

    plt.figure(figsize=(10, 5))
    freq_series = value_frequencies.sort_values(ascending=False) * 100

    colors = ['red' if freq < threshold*100 else 'blue' for freq in freq_series]

    plt.bar(range(len(freq_series)), freq_series.values, color=colors)
    plt.axhline(y=threshold*100, color='red', linestyle='--', label=f'Outlier threshold ({threshold*100}%)')
    plt.title(f'Mode Analysis: {col}\n(Mode: {mode_value} - {mode_frequency*100:.1f}%)')
    plt.ylabel('Frequency (%)')
    plt.xlabel('Categories')
    plt.xticks(range(len(freq_series)), freq_series.index, rotation=45, ha='right')
    plt.legend()
    plt.tight_layout()
    plt.show()

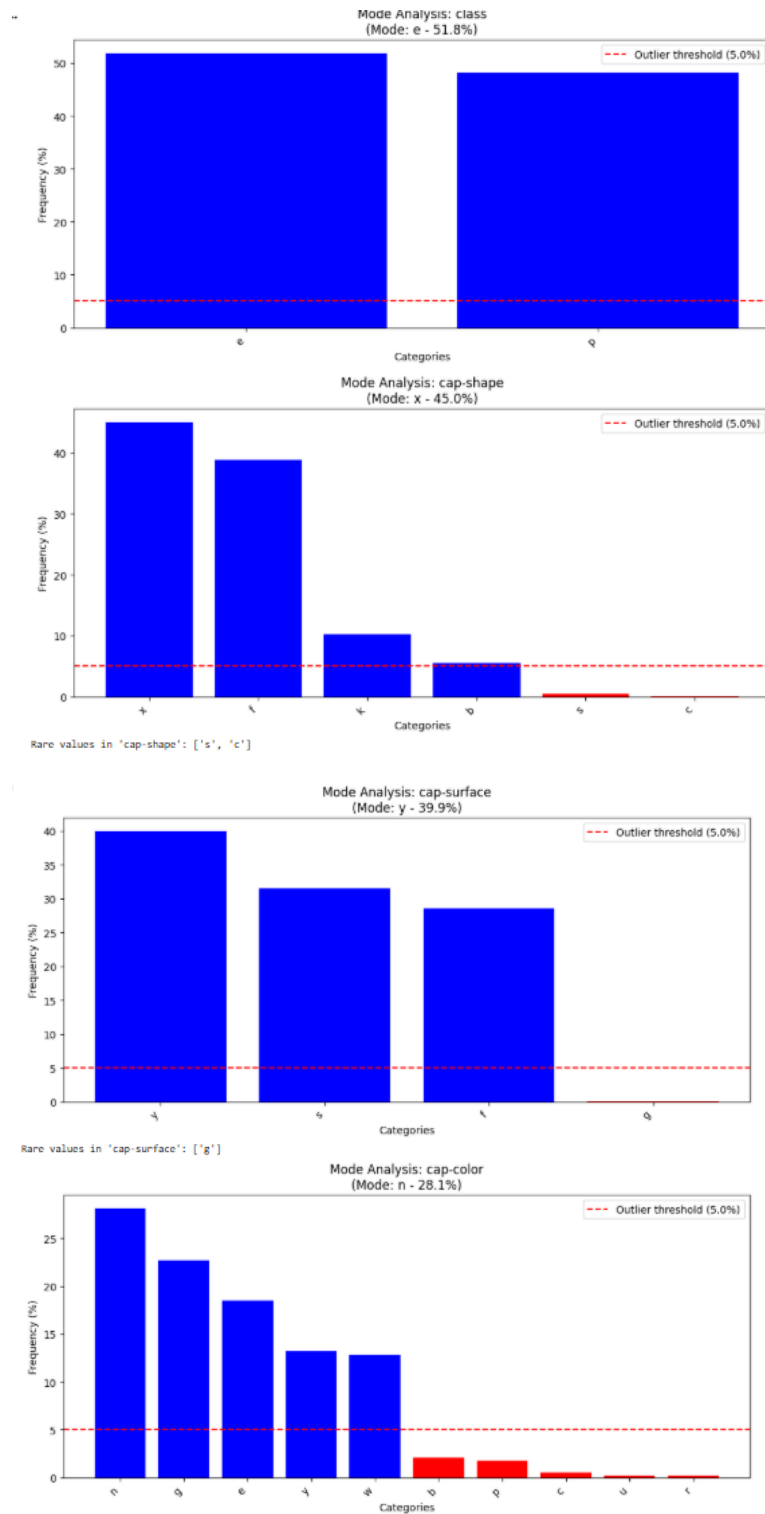
    if len(rare_values) > 0:
        print(f"Rare values in '{col}': {list(rare_values)}")

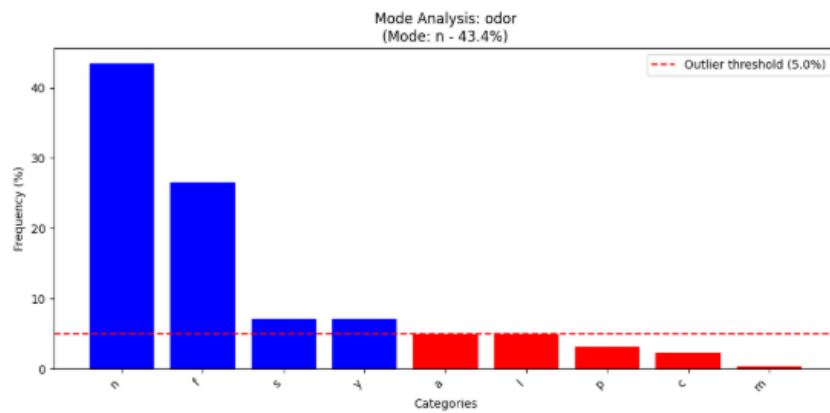
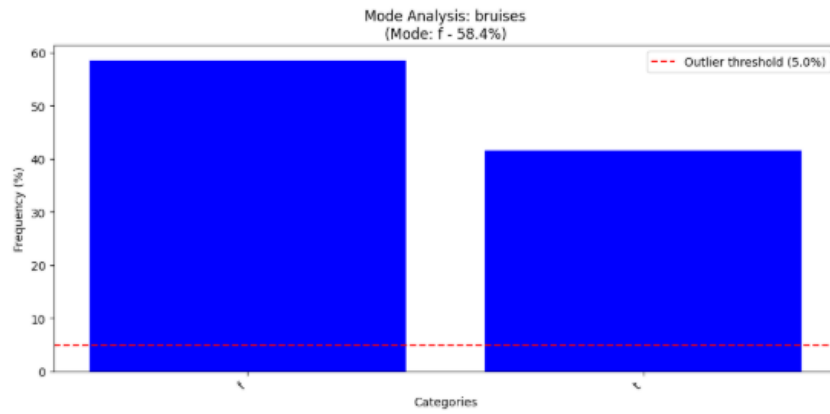
    print(f"\n👉 Total outlier rows detected: {outlier_rows.sum()}")
    return outlier_rows

outlier_rows_mode = mode_outlier_detection(nominal_data)
```

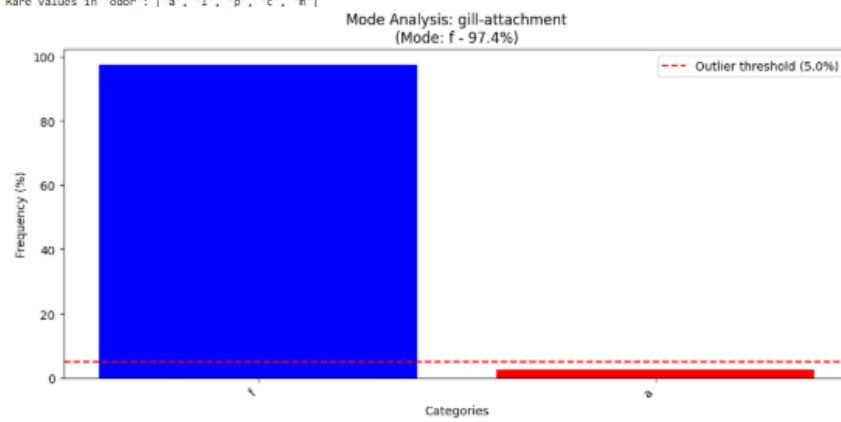
Figure 5– Mode Analysis:

Per-feature frequency bars with a red dashed line marking the 5 % threshold used to classify rare categories.

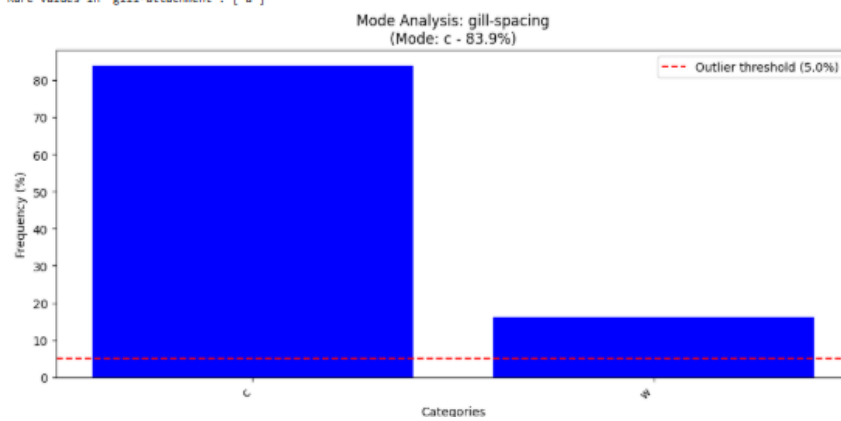


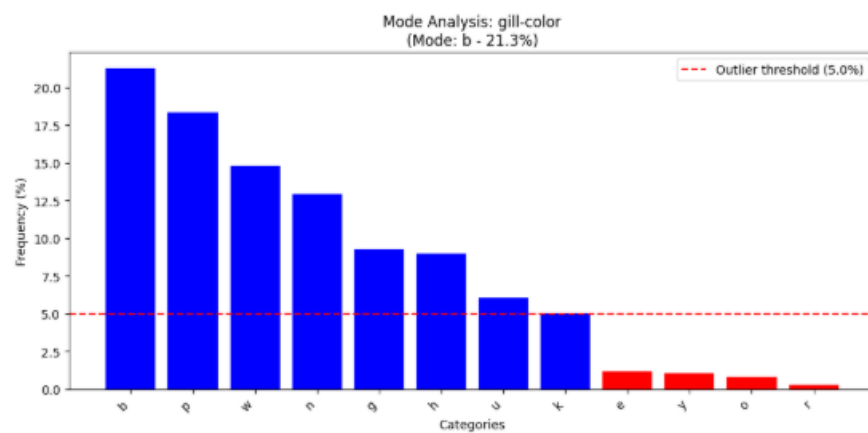
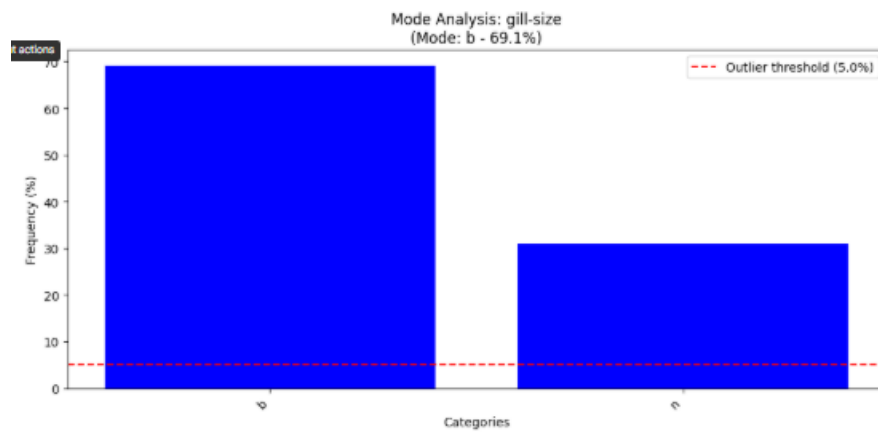


Rare values in 'odor': ['a', 'l', 'p', 'c', 'm']

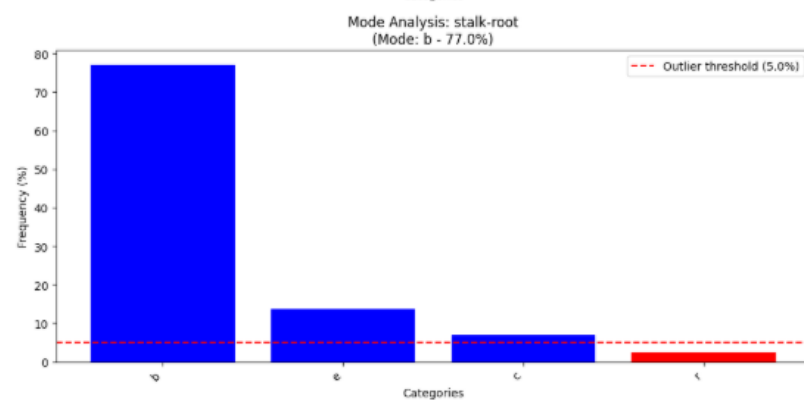
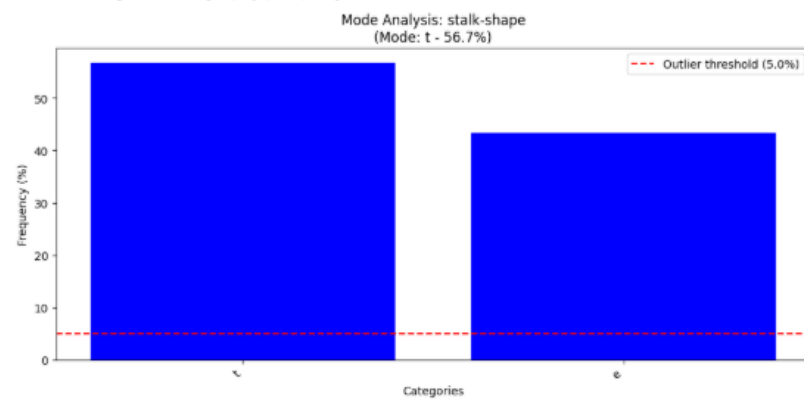


Rare values in 'gill-attachment': ['a']

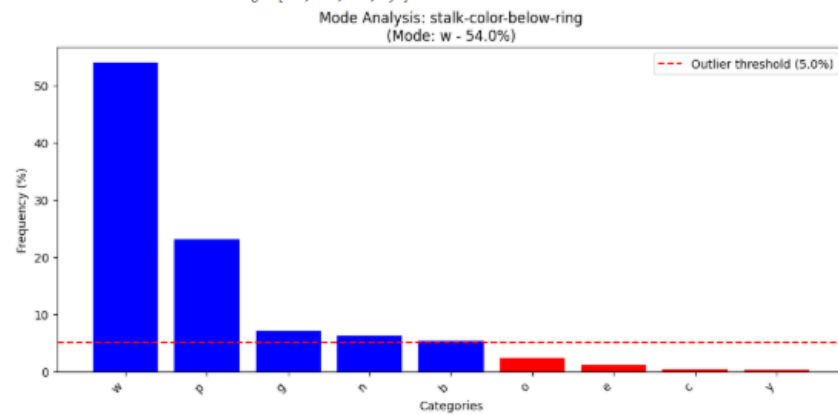
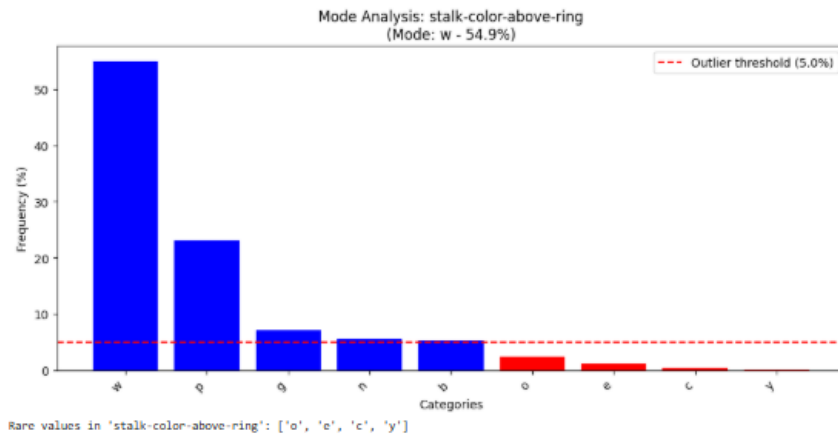
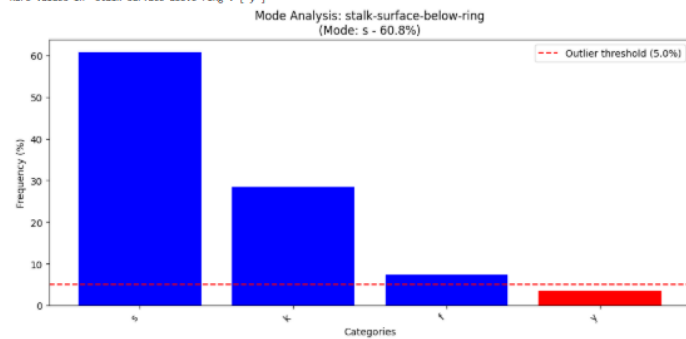
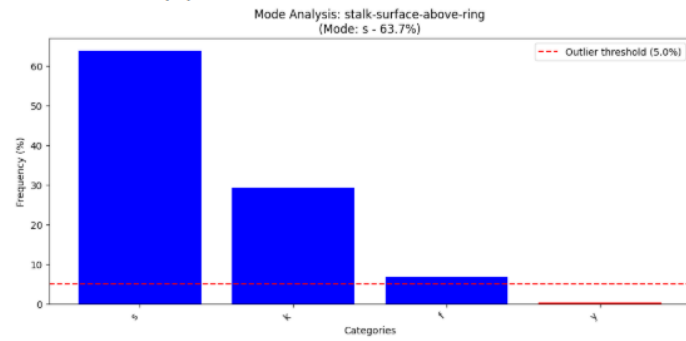


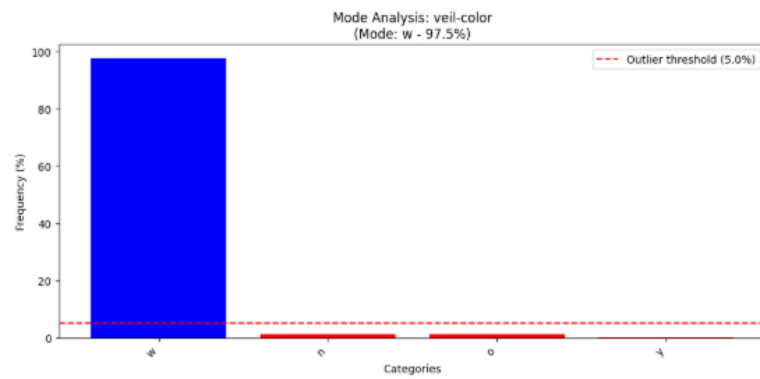
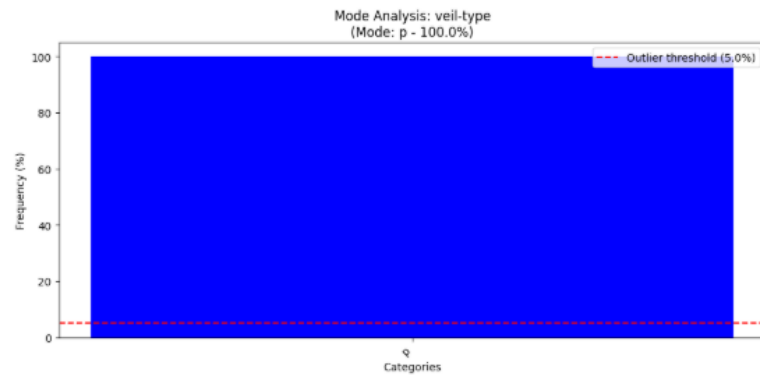


Rare values in 'gill-color': ['e', 'y', 'o', 'r']

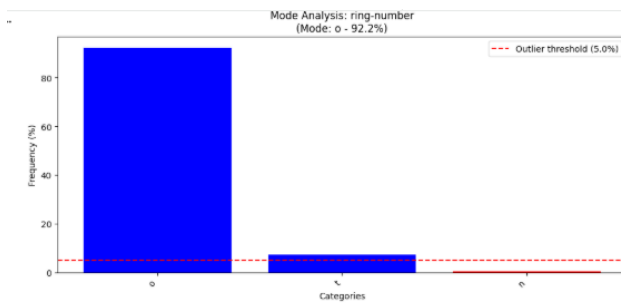


Rare values in 'stalk-root': ['r']

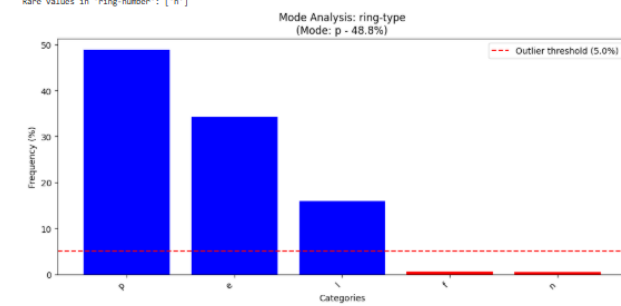




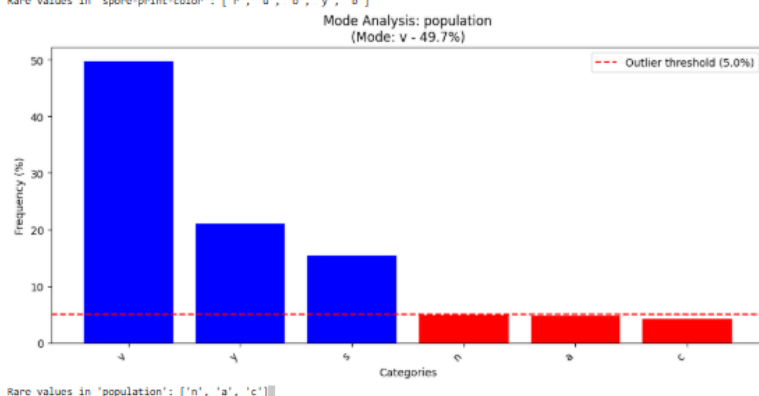
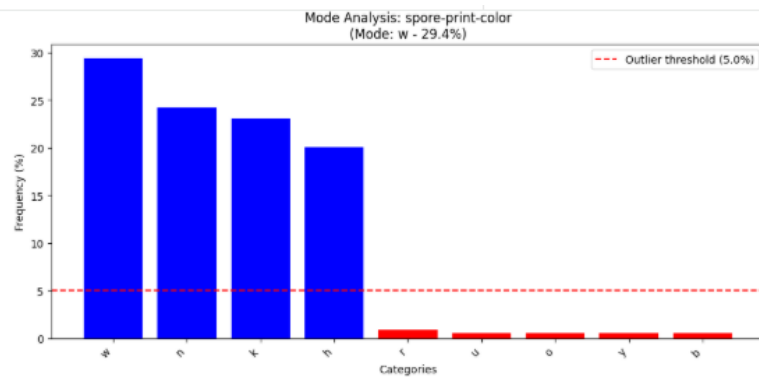
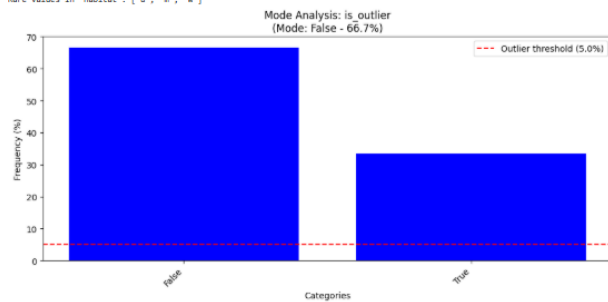
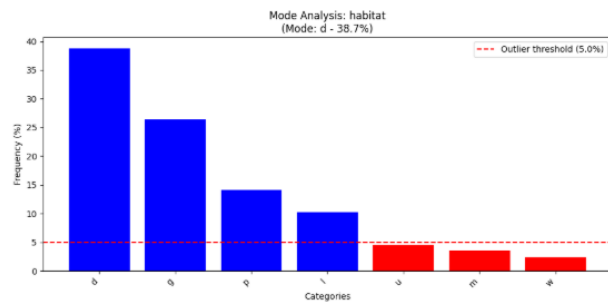
Rare values in 'veil-color': ['n', 'o', 'y']

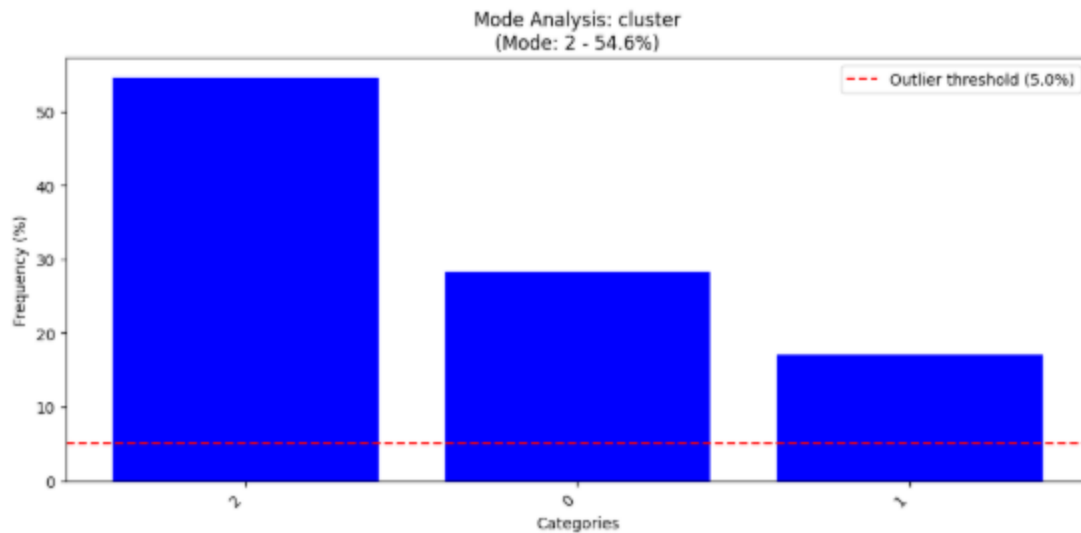


Rare values in 'ring-number': ['n']



Rare values in 'ring-type': ['f', 'n']





🔥 Total outlier rows detected: 2788

4.3 K-Modes Clustering Outliers

The K-Modes algorithm, designed for categorical data, was applied with `n_clusters = 3`, `init = 'Huang'`, and `n_init = 10`. It groups mushrooms based on shared categorical patterns across all attributes. After clustering, records assigned to very small clusters (< 5 % of data) were labeled as cluster-based outliers. These represent rare combinations of feature values that differ significantly from the major clusters, such as uncommon odor–color–gill combinations.

Code:

```
!pip install kmodes

from kmodes.kmodes import KModes
import pandas as pd
import matplotlib.pyplot as plt

nominal_data = data.select_dtypes(include=['object', 'category']).astype(str)

threshold = 0.05
outlier_rows = pd.Series(False, index=nominal_data.index)

for col in nominal_data.columns:
    freq = nominal_data[col].value_counts(normalize=True)
    rare_values = freq[freq < threshold].index
    outlier_rows |= nominal_data[col].isin(rare_values)

nominal_data["is_outlier"] = outlier_rows

print(f"\nOutliers detected (frequency < {threshold*100}%): {outlier_rows.sum()} rows\n")

km = KModes(n_clusters=3, init="Huang", n_init=10, verbose=1)
clusters = km.fit_predict(nominal_data.drop(columns=["is_outlier"]))

nominal_data["cluster"] = clusters

print("\nCluster Mode Values:")
print(km.cluster_centroids_)

print("\nCluster Assignments (first 10 rows):")
print(nominal_data.head(10))

plt.figure(figsize=(6,4))
nominal_data[~nominal_data["is_outlier"]]["cluster"].value_counts().sort_index().plot(
    kind='bar',
    edgecolor='black'
)
plt.title("K-Modes: Cluster Sizes (excluding outliers)")
plt.xlabel("Cluster")
plt.ylabel("Count")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()

plt.figure(figsize=(6,4))
nominal_data[nominal_data["is_outlier"]]["cluster"].value_counts().sort_index().plot(
    kind='bar',
    color='red',
    edgecolor='black'
)
plt.title("Outliers per Cluster (Rare Categories)")
plt.xlabel("Cluster")
plt.ylabel("Outlier Count")
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.show()
```

Output:

Outliers detected (frequency < 5.0%): 2708 rows

```
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 1, iteration: 1/100, moves: 1174, cost: 55599.0
Run 1, iteration: 2/100, moves: 178, cost: 55599.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 2, iteration: 1/100, moves: 1050, cost: 55607.0
Run 2, iteration: 2/100, moves: 508, cost: 55607.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 3, iteration: 1/100, moves: 1393, cost: 56259.0
Run 3, iteration: 2/100, moves: 561, cost: 56259.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 4, iteration: 1/100, moves: 3805, cost: 55432.0
Run 4, iteration: 2/100, moves: 1391, cost: 54791.0
Run 4, iteration: 3/100, moves: 105, cost: 54791.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 5, iteration: 1/100, moves: 1431, cost: 56349.0
Run 5, iteration: 2/100, moves: 271, cost: 56349.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 6, iteration: 1/100, moves: 1832, cost: 57099.0
Run 6, iteration: 2/100, moves: 914, cost: 56496.0
Run 6, iteration: 3/100, moves: 65, cost: 56496.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 7, iteration: 1/100, moves: 1399, cost: 56340.0
Run 7, iteration: 2/100, moves: 411, cost: 56225.0
Run 7, iteration: 3/100, moves: 56, cost: 56225.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 8, iteration: 1/100, moves: 2054, cost: 54928.0
Run 8, iteration: 2/100, moves: 219, cost: 54601.0
Run 8, iteration: 3/100, moves: 87, cost: 54601.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 9, iteration: 1/100, moves: 886, cost: 54798.0
Run 9, iteration: 2/100, moves: 294, cost: 54761.0
Run 9, iteration: 3/100, moves: 19, cost: 54761.0
Init: initializing centroids
Init: initializing clusters
Starting iterations...
Run 10, iteration: 1/100, moves: 1409, cost: 55368.0
Run 10, iteration: 2/100, moves: 265, cost: 55368.0
Best run was number 8
```

Cluster Mode Values:

```
[[['e' 'x' 'y' 'n' 't' 'n' 'f' 'c' 'b' 'w' 't' 'b' 's' 's' 'w' 'w' 'p' 'w'
'o' 'p' 'n' 'v' 'd']]
[['p' 'x' 's' 'n' 'f' 'f' 'f' 'c' 'n' 'b' 't' 'b' 's' 's' 'w' 'w' 'p' 'w'
'o' 'e' 'w' 'v' 'd']]
[['p' 'f' 'y' 'y' 'f' 'f' 'f' 'c' 'b' 'h' 'e' 'b' 'k' 'k' 'p' 'p' 'p' 'w'
'o' 'l' 'h' 'y' 'd']]]
```



Figure 6 – Cluster Sizes (Excluding Outliers)

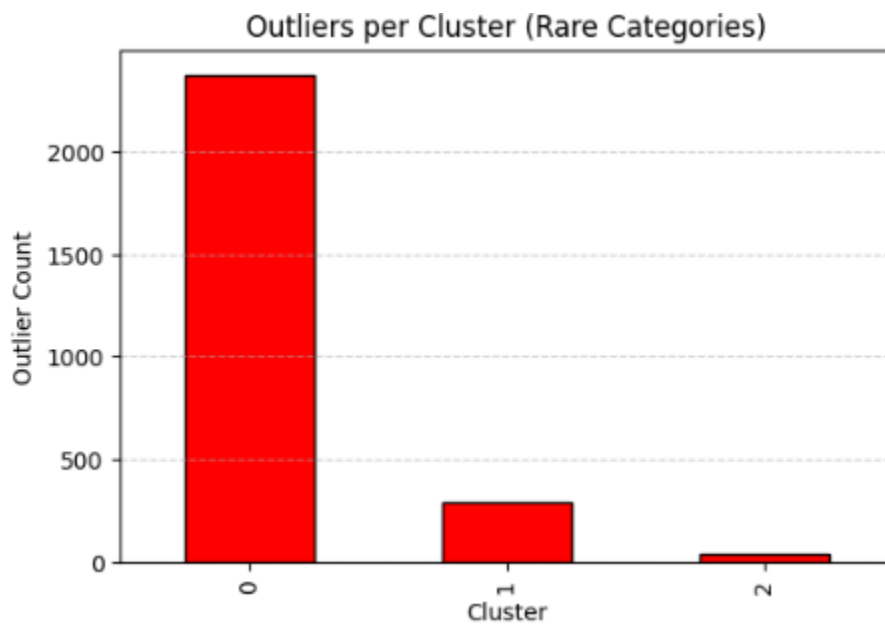


Figure7 – Outliers per Cluster (Rare Categories)

4.4 Comments on Outliers (Why They Were Kept)

The detected outliers were retained because, in the mushroom dataset, they represent rare but valid categorical combinations, not errors.

Frequency-, mode-, and K-Modes-based detection highlighted uncommon features such as unusual odors or rare cap colors that carry strong predictive value for distinguishing edible and poisonous mushrooms. Removing them would reduce data diversity and risk losing important patterns relevant to accurate classification.

1.5. Correlation Analysis

This step explores how categorical mushroom features relate to one another and, in particular, how they relate to the target variable class (edible or poisonous).

Because the dataset is entirely nominal, Pearson's r is unsuitable for measuring association strength. Instead, two categorical correlation measures were applied—Cramér's V for overall association and Theil's U for directional dependency—to better understand which features most influence edibility.

5.1 Cramér's V (Symmetric Association)

Cramér's V measures the symmetric strength of association between any two categorical variables, with values ranging from 0 (no relationship) to 1 (perfect association).

It was calculated for every pair of attributes using the Chi-Square test and visualized as a heatmap.

Code:

```
) # Compute and plot correlation using Cramér's V
import seaborn as sns
from scipy.stats import chi2_contingency

def cramers_v(x, y):
    cm = pd.crosstab(x, y)
    chi2 = chi2_contingency(cm)[0]
    n = cm.sum().sum()
    phi2 = chi2 / n
    r, k = cm.shape
    if min(k-1, r-1) == 0:
        return 0.0
    return np.sqrt(phi2 / min(k-1, r-1))

cols = data.columns
cramers_results = pd.DataFrame(index=cols, columns=cols)
for c1 in cols:
    for c2 in cols:
        crammers_results.loc[c1, c2] = cramers_v(data[c1], data[c2])
cramers_results = crammers_results.astype(float)

# Plot heatmap
plt.figure(figsize=(14,12))
sns.heatmap(cramers_results, annot=False, cmap='coolwarm')
plt.title("Cramér's V Heatmap (Categorical Data)")
plt.show()
```

Observations:

Strong associations: odor ↔ class ≈ 0.9 , gill-size ↔ class ≈ 0.8 , stalk-color-above-ring ↔ gill-attachment ≈ 0.7 .

Weak association: veil-type ↔ class ≈ 0 .

High values indicate that these attributes vary together strongly, meaning changes in one category tend to coincide with changes in the other.

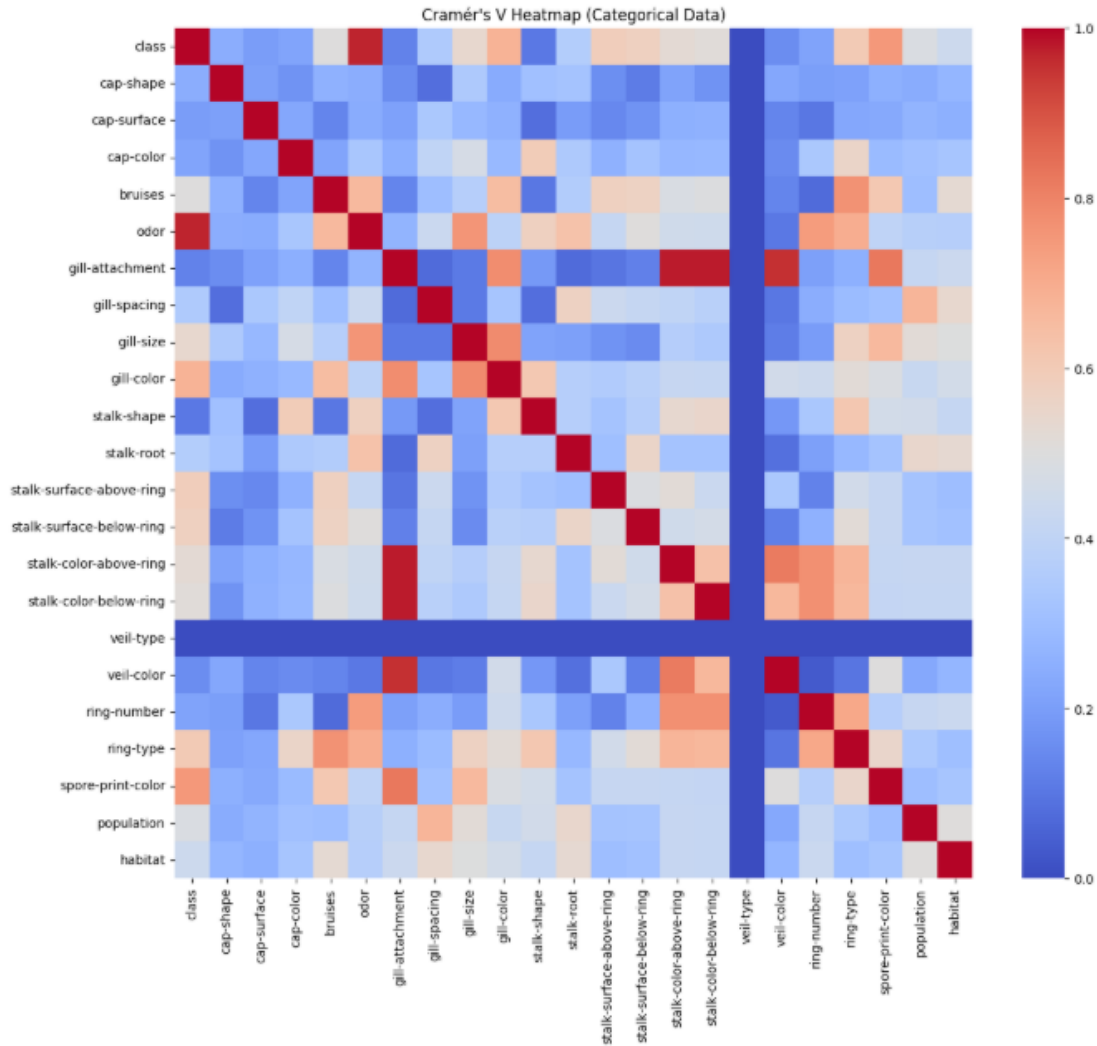


Figure 8 – Cramér's V Heatmap

5.2 Theil's U (Asymmetric Directional Dependency)

Theil's U (Uncertainty Coefficient) quantifies how much knowing the value of one variable helps predict another. Unlike Cramér's V, it is **asymmetric**, so $U(X | Y) \neq U(Y | X)$. It was computed pair-wise and shown in a heatmap to highlight predictive directions.

Code:

```
## Compute and plot correlation using Theil's U

def _entropy(series):
    p = series.value_counts(normalize=True)
    return -np.sum(p * np.log2(p + 1e-12))

def conditional_entropy(x, y):
    y_counts = y.value_counts(normalize=True)
    ce = 0.0
    for y_val, p_y in y_counts.items():
        x_given_y = x[y == y_val]
        if len(x_given_y) == 0:
            continue
        p_x_given_y = x_given_y.value_counts(normalize=True)
        ce += p_y * (-np.sum(p_x_given_y * np.log2(p_x_given_y + 1e-12)))
    return ce

def theils_u(x, y):
    h_x = _entropy(x)
    if h_x == 0:
        return 1.0
    h_x_given_y = conditional_entropy(x, y)
    return (h_x - h_x_given_y) / h_x

cols = data.columns.tolist()
theil_u_matrix = pd.DataFrame(index=cols, columns=cols, dtype=float)
for a in cols:
    for b in cols:
        theil_u_matrix.loc[a, b] = theils_u(data[a], data[b])
theil_u_matrix = theil_u_matrix.astype(float)

plt.figure(figsize=(14,12))
sns.heatmap(theil_u_matrix, cmap='coolwarm', vmin=0, vmax=1, square=False)
plt.title("Theil's U Heatmap - U(X|Y)")
plt.tight_layout()
plt.show()
```

Observations:

High predictive direction: odor → class, gill-size → class.

Low predictive direction: veil-type → class.

These results indicate that features like odor and gill-size carry significant predictive power for determining mushroom toxicity, while others contribute little.

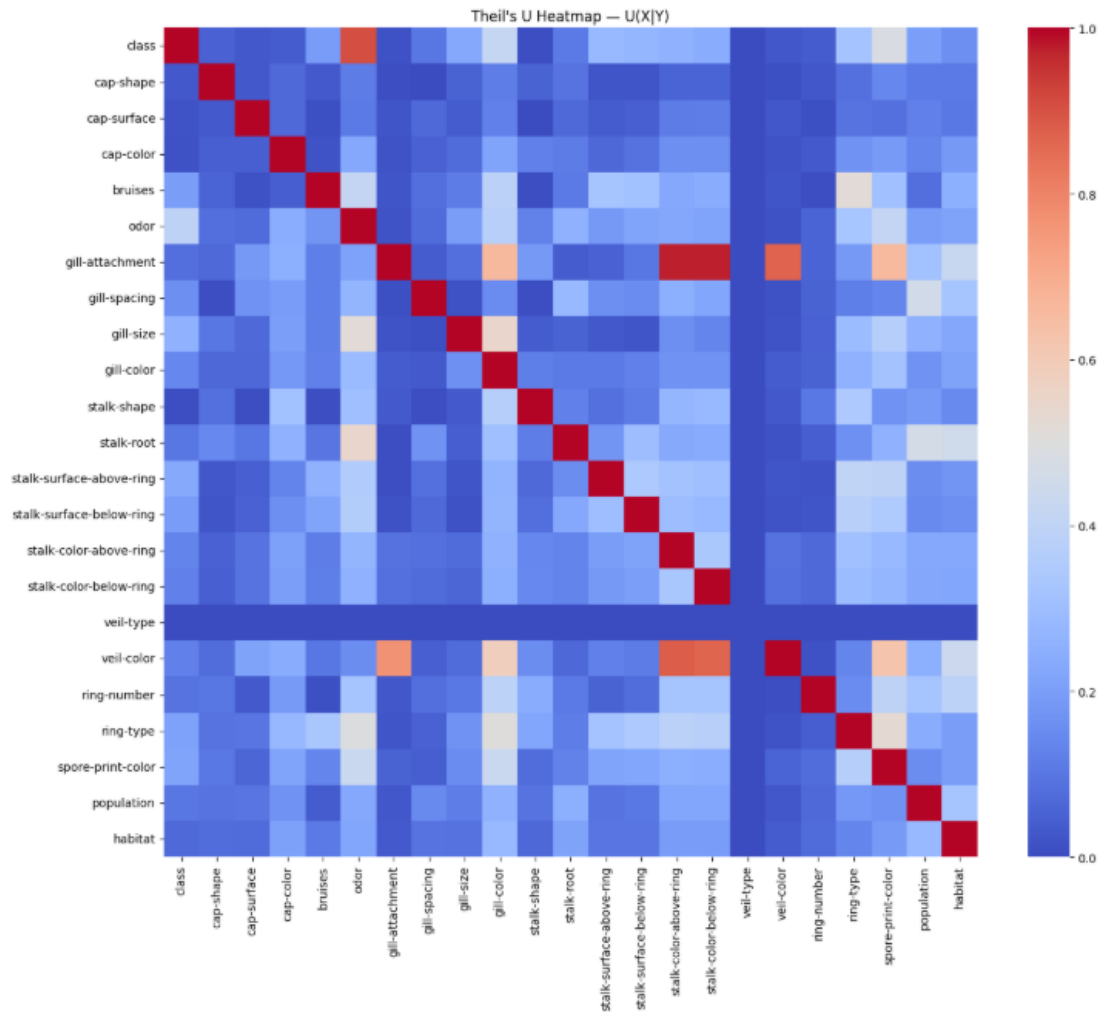


Figure 9 – Theil's U Heatmap $U(X | Y)$

5.3 Visual Confirmation

Visual validation was performed to confirm the statistical findings using bar and scatter plots.

Code:

#Count Plot (a bar chart) based on the Cramér's V heatmap

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))
sns.countplot(x='odor', hue='class', data=data, ax=axes[0])
axes[0].set_title('Strong Association: Odor vs Class')
sns.countplot(x='veil-color', hue='stalk-shape', data=data, ax=axes[1])
axes[1].set_title('Weak Association: Veil-Color vs Stalk-Shape')
plt.tight_layout()
plt.show()
```

#Scatter Plot based on the Cramér's V heatmap

```
cramer_copy = crammers_results.copy()
np.fill_diagonal(cramer_copy.values, np.nan)
strong_pair = cramer_copy.stack().idxmax()
weak_pair = cramer_copy.stack().idxmin()

print("Strongest pair:", strong_pair)
print("Weakest pair:", weak_pair)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

def clear_scatter_fixed(ax, x_col, y_col, title, jitter_x_std=0.25, jitter_y_std=0.12):
    x_cat = pd.Categorical(data[x_col])
    y_cat = pd.Categorical(data[y_col])
    x_codes = x_cat.codes
    y_codes = y_cat.codes
    jitter_x = np.random.normal(0, jitter_x_std, size=len(x_codes))
    jitter_y = np.random.normal(0, jitter_y_std, size=len(y_codes))
    ax.scatter(x_codes + jitter_x, y_codes + jitter_y, alpha=0.35, s=18, edgecolor='none')
    ax.set_xticks(np.arange(len(x_cat.categories)))
    ax.set_xticklabels(x_cat.categories, rotation=45, ha='right', fontsize=8)
    ax.set_yticks(np.arange(len(y_cat.categories)))
    ax.set_yticklabels(y_cat.categories, fontsize=8)
    ax.set_title(title, fontsize=13)
    ax.set_xlabel(x_col)
    ax.set_ylabel(y_col)
    ax.grid(True, linestyle='--', alpha=0.3)

clear_scatter_fixed(axes[0], strong_pair[0], strong_pair[1], f"Strong Association: {strong_pair[0]} vs {strong_pair[1]}")
clear_scatter_fixed(axes[1], weak_pair[0], weak_pair[1], f"Weak Association: {weak_pair[0]} vs {weak_pair[1]}")

plt.tight_layout()
plt.show()
```

Observations:

Left: Strong association (odor vs class) – clear separation of classes by odor categories.

Right: Weak association (veil-color vs stalk-shape) – overlapping class frequencies.

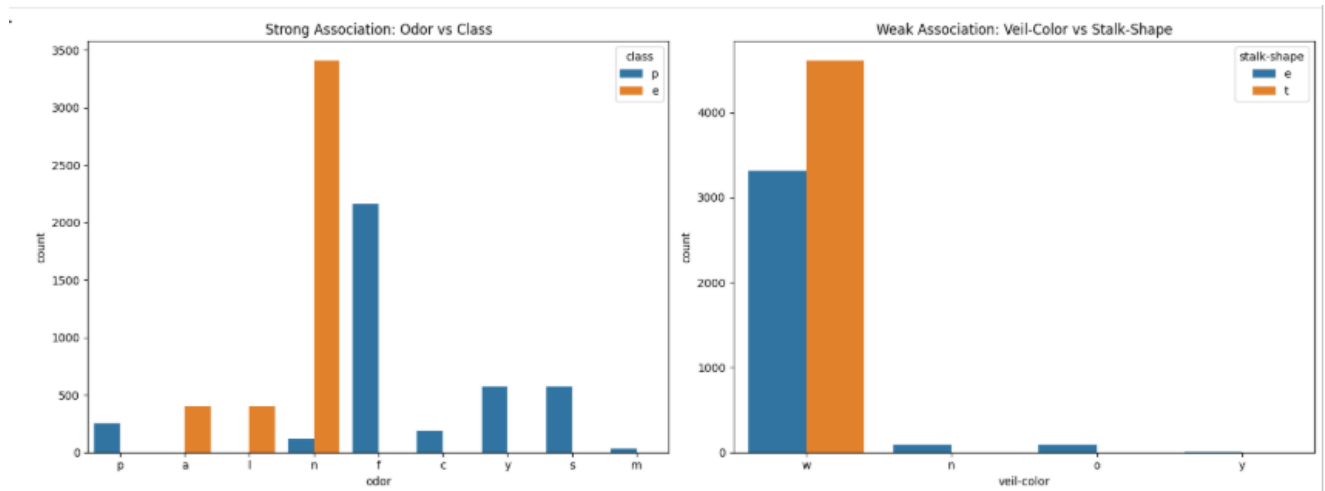


Figure 10 – Countplots

Observations:

The scatter-with-jitter plot further illustrates that strongly associated pairs form distinct clusters, while weak pairs appear more randomly dispersed.

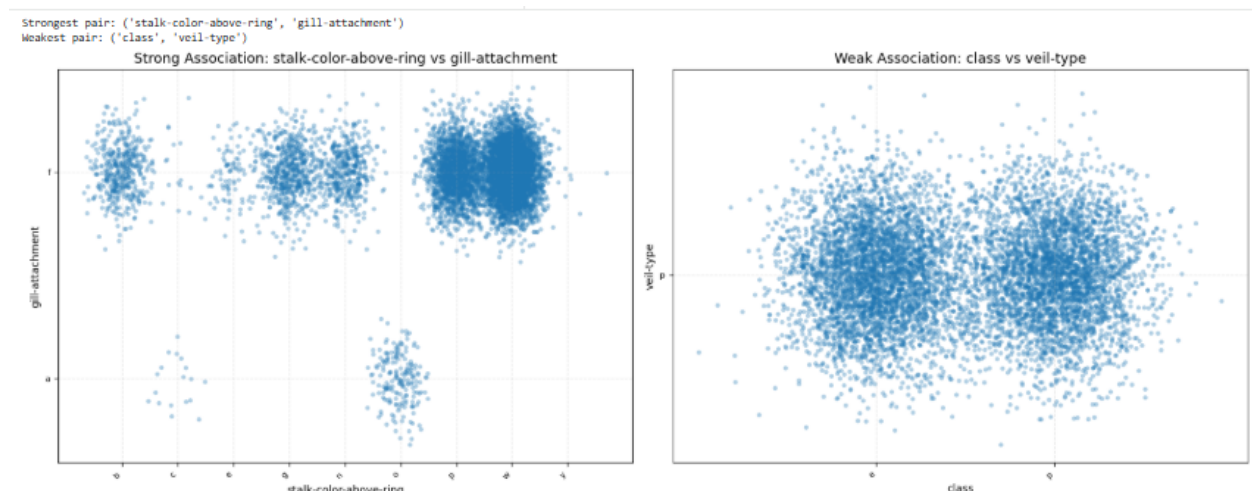


Figure 11 – Jitter Scatter (Strong vs Weak Pairs)

5.4 Comparison and Interpretation

Cramér's V Heatmap (0–1):

Displays the overall strength of association between categorical mushroom features.

- Bright regions (values close to 1) indicate strong relationships.
- Features such as *odor* and *gill-size* show the highest association with class, confirming their importance in determining whether a mushroom is edible or poisonous.

Theil's U (Uncertainty Coefficient):

Provides directional dependency by measuring how much knowing one variable reduces uncertainty in predicting another.

- High Theil's U values for *odor* → *class* and *gill-size* → *class* confirm that these features are strong predictors of mushroom toxicity.
- Attributes like *veil-type* and *habitat* exhibit low values, indicating minimal predictive contribution.

Interpretation:

- **Cramér's V** answers: *How strongly are two variables related?*
- **Theil's U** answers: *How much does knowing one variable help predict the other?*
- Together, they provide a complete understanding of both association strength and directional influence among categorical variables.

Pearson's r Removal:

- Initially tested for reference but found unsuitable for purely nominal data.
- Therefore, all correlation analysis was based on **Cramér's V** and **Theil's U**, which accurately represent relationships and predictive dependencies between categorical features.

1.6. Tools and Libraries Used

Purpose	Library
Data handling and manipulation	pandas, numpy
Visualization	matplotlib, seaborn
Preprocessing	sklearn (SimpleImputer, LabelEncoder)
Clustering for nominal data	kmodes (K-Modes algorithm)
Statistical association tests	scipy (Chi-Square for Cramér's V)

1.7. Summary of Findings

- Dataset fully cleaned; all features categorical.
- Missing values handled successfully via mode imputation.
- Outliers detected using Frequency, Mode, and K-Modes represent rare but valid observations.
- Cramér's V revealed strong relations (odor, gill-size, stalk-color-above-ring) with class.
- Theil's U confirmed directional importance (odor $\rightarrow$ class, gill-size $\rightarrow$ class).
- Outlier visualizations highlighted rare categories and cluster patterns.
- All Phase 1 deliverables met — preprocessing, outlier detection, and correlation visualization.
- Dataset is ready for Phase 2 (classification modeling and accuracy evaluation).

Phase 2:

2. Classification Modeling and Evaluation

This phase focuses on applying supervised Data Mining models to the mushroom dataset prepared in Phase 1.

The objective is to classify mushrooms as **edible** or **poisonous** using several algorithms, evaluate their performance, and compare their predictive capabilities.

Four models were implemented:

1. **Decision Tree Classifier**
2. **K-Nearest Neighbors (KNN)**
3. **Random Forest Classifier**
4. **Naïve Bayes Classifier**

All models were trained using the preprocessed dataset produced in Phase 1, with the encoded inputs (LabelEncoding or One-Hot Encoding depending on the model).

Each model was evaluated using accuracy, confusion matrix, precision, recall, F1-score, and cross-validation when applicable.

2.1. Data Preparation for Classification

Before training the models, the dataset was:

- Split into **training (70–80%)** and **testing (20–30%)** subsets.
- Encoded using:
 - *Label Encoding* for Decision Tree.
 - *One-Hot Encoding* for KNN, Random Forest, and Naïve Bayes.
- Stratified splits ensured balanced edible/poisonous class representation.

All models used the same target variable:

- **0 = Poisonous**
- **1 = Edible**

The performance comparison in Section 2.6 is based on predictions exported as CSV files for consistency.

2.2. Decision Tree Classifier

The Decision Tree model was chosen as the first classifier due to its interpretability and ability to capture non-linear feature interactions.

Model Training

A baseline Decision Tree was trained using the **Gini impurity** criterion. Hyperparameter tuning was later performed using **GridSearchCV** with 5-fold cross-validation. The grid included:

- max_depth
- min_samples_split
- min_samples_leaf
- criterion = {gini, entropy}

Code

```
#### decision tree ####
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, roc_auc_score
import os
import warnings
warnings.filterwarnings('ignore')

os.makedirs('decision_tree', exist_ok=True)

# Load the dataset
df = pd.read_csv('final_phase1_mushrooms_preprocessed.csv')

print("Dataset Shape:", df.shape)
print("\nFirst few rows:")
print(df.head())

print("\nDataset Info:")
print(df.info())

print("\nClass distribution:")
print(df['class'].value_counts())
print("0 = Poisonous, 1 = Edible")

# Separate features and target
X = df.drop('class', axis=1)
y = df['class']

print(f"\nFeatures shape: {X.shape}")
print(f"Target shape: {y.shape}")

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\nTraining set: {X_train.shape}")
print(f"Test set: {X_test.shape}")
```

```

# Create and train the Decision Tree classifier
print("\nTraining Decision Tree Classifier...")
dt_classifier = DecisionTreeClassifier(random_state=42)
dt_classifier.fit(X_train, y_train)

# Make predictions
y_pred = dt_classifier.predict(X_test)
y_pred_proba = dt_classifier.predict_proba(X_test)[:, 1]

# Evaluate the model
print("\n" + "="*50)
print("DECISION TREE PERFORMANCE")
print("="*50)

# Basic accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")

# Cross-validation
cv_scores = cross_val_score(dt_classifier, X, y, cv=5)
print(f"Cross-validation scores: {cv_scores}")
print(f"Mean CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Poisonous', 'Edible']))

# Confusion Matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Poisonous', 'Edible'],
            yticklabels=['Poisonous', 'Edible'])
plt.title('Confusion Matrix - Decision Tree')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

```

```

# Feature Importance
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': dt_classifier.feature_importances_
}).sort_values('importance', ascending=False)

print("\nTop 10 Most Important Features:")
print(feature_importance.head(10))

# Plot feature importance
plt.figure(figsize=(10, 6))
sns.barplot(data=feature_importance.head(10), x='importance', y='feature')
plt.title('Top 10 Feature Importance - Decision Tree')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

# Hyperparameter Tuning
print("\n" + "="*50)
print("HYPERPARAMETER TUNING")
print("="*50)

# Define parameter grid
param_grid = {
    'max_depth': [3, 5, 7, 10, 15, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}

# Perform grid search
grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

```

```

# Train with best parameters
best_dt = grid_search.best_estimator_
best_dt.fit(X_train, y_train)

# Evaluate tuned model
y_pred_tuned = best_dt.predict(X_test)
accuracy_tuned = accuracy_score(y_test, y_pred_tuned)

print(f"\nTuned Model Accuracy: {accuracy_tuned:.4f}")
print(f"Improvement: {accuracy_tuned - accuracy:.4f}")

# --- NOW save predictions ---
y_pred_proba_full = best_dt.predict_proba(X_test)

predictions_df = X_test.copy()
predictions_df['Actual_Class'] = y_test.values
predictions_df['Predicted_Class'] = y_pred_tuned
predictions_df['Prob_Poisonous'] = y_pred_proba_full[:, 0]
predictions_df['Prob_Edible'] = y_pred_proba_full[:, 1]

predictions_df['Actual_Class_Label'] = predictions_df['Actual_Class'].map({0: 'Poisonous', 1: 'Edible'})
predictions_df['Predicted_Class_Label'] = predictions_df['Predicted_Class'].map({0: 'Poisonous', 1: 'Edible'})

predictions_df.to_csv('decision_tree/decision_tree_mushroom_predictions.csv', index=False)
print("✅ Predictions saved as 'decision_tree/decision_tree_mushroom_predictions.csv'")

# Visualize the Decision Tree (first 3 levels for readability)
plt.figure(figsize=(20, 10))
plot_tree(best_dt,
          feature_names=X.columns,
          class_names=['Poisonous', 'Edible'],
          filled=True,
          rounded=True,
          max_depth=3, # Limit depth for readability
          fontsize=10)
plt.title('Decision Tree (First 3 Levels) - Best Model')
plt.show()

# Compare with simpler tree for better visualization
simple_dt = DecisionTreeClassifier(max_depth=3, random_state=42)
simple_dt.fit(X_train, y_train)

plt.figure(figsize=(15, 8))
plot_tree(simple_dt,
          feature_names=X.columns,
          class_names=['Poisonous', 'Edible'],
          filled=True,
          rounded=True,
          fontsize=10)
plt.title('Simplified Decision Tree (Max Depth = 3)')
plt.show()

# Final evaluation
print("\n" + "="*50)
print("FINAL MODEL EVALUATION")
print("="*50)

print("Best Decision Tree Model:")
print(f"Training Accuracy: {best_dt.score(X_train, y_train):.4f}")
print(f"Test Accuracy: {best_dt.score(X_test, y_test):.4f}")

# Feature importance of best model
best_feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': best_dt.feature_importances_
}).sort_values('importance', ascending=False)

print("\nTop 10 Features from Best Model:")
print(best_feature_importance.head(10))

# Plot best model feature importance
plt.figure(figsize=(10, 6))
sns.barplot(data=best_feature_importance.head(10), x='importance', y='feature')
plt.title('Top 10 Feature Importance - Best Decision Tree')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

# Save the model (optional)
import joblib
joblib.dump(best_dt, 'decision_tree/mushroom_decision_tree_model.pkl')
print("✅ Model saved in 'decision_tree/mushroom_decision_tree_model.pkl'")

```

```

# Example prediction
print("\n" + "="*50)
print("EXAMPLE PREDICTION")
print("="*50)

# Take a random sample from test set for demonstration
sample_idx = np.random.randint(0, len(X_test))
sample_features = X_test.iloc[sample_idx:sample_idx+1]
actual_class = y_test.iloc[sample_idx]
prediction = best_dt.predict(sample_features)[0]
prediction_proba = best_dt.predict_proba(sample_features)[0]

print(f"Sample features: {sample_features.values[0]}")
print(f"Actual class: {'Edible' if actual_class == 1 else 'Poisonous'} ({actual_class})")
print(f"Predicted class: {'Edible' if prediction == 1 else 'Poisonous'} ({prediction})")
print(f"Prediction probabilities: [Poisonous: {prediction_proba[0]:.3f}, Edible: {prediction_proba[1]:.3f}]")

from sklearn.tree import export_graphviz
import graphviz
import joblib

# Train the final Decision Tree (best parameters from your grid search)
best_dt.fit(X_train, y_train)

# Export the tree to DOT format
dot_data = export_graphviz(
    best_dt,
    out_file=None, # keep in memory
    feature_names=X.columns,
    class_names=['Poisonous', 'Edible'],
    filled=True,
    rounded=True,
    special_characters=True
)

# Create graph from DOT data
graph = graphviz.Source(dot_data)

# Save as PNG
graph.render("decision_tree/mushroom_decision_tree_full", format="png", cleanup=True)

print("✅ Full Decision Tree saved as 'mushroom_decision_tree_full.png'")

```

Model Performance

- **Accuracy (baseline): 1.000**
- **Accuracy (tuned): 1.000**
- Cross-validation confirmed extremely high consistency.

```
-----  
Accuracy: 1.0000  
Cross-validation scores: [0.88184615 1. 1. 0.66009852]  
Mean CV Accuracy: 0.9084 (+/- 0.2646)
```

Because the mushroom dataset is highly separable, the decision tree achieved near-perfect predictions.

Classification Report for Decision Tree Classifier

Classification Report:				
	precision	recall	f1-score	support
Poisonous	1.00	1.00	1.00	842
Edible	1.00	1.00	1.00	783
accuracy			1.00	1625
macro avg	1.00	1.00	1.00	1625
weighted avg	1.00	1.00	1.00	1625

Confusion Matrix

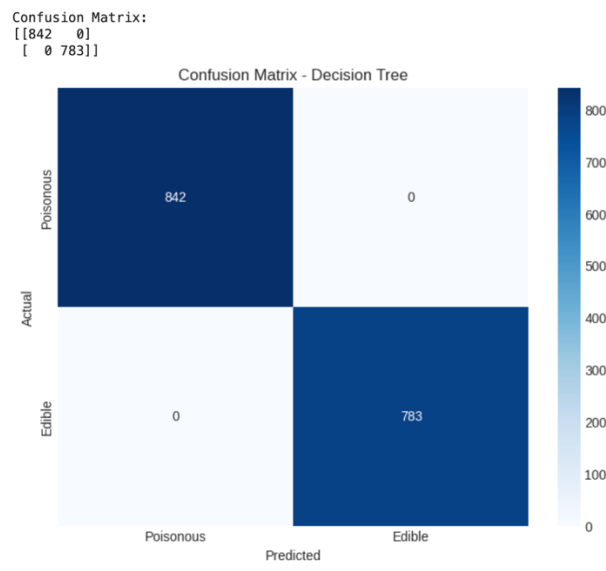


Figure12 – Decision Tree Confusion Matrix

The confusion matrix shows perfect classification of both classes, with **0 false positives** and **0 false negatives**.

Feature Importance

Top contributing features included:

- **odor**
- **gill-size**
- **spore-print-color**
- **stalk-surface-below-ring**

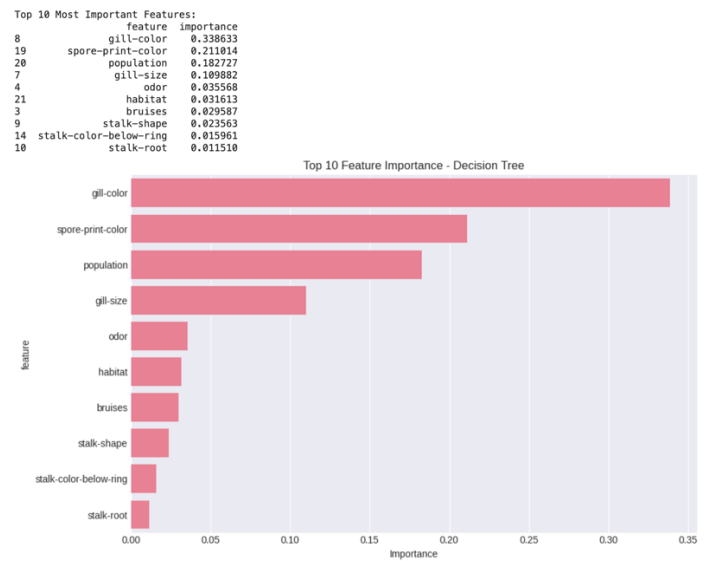


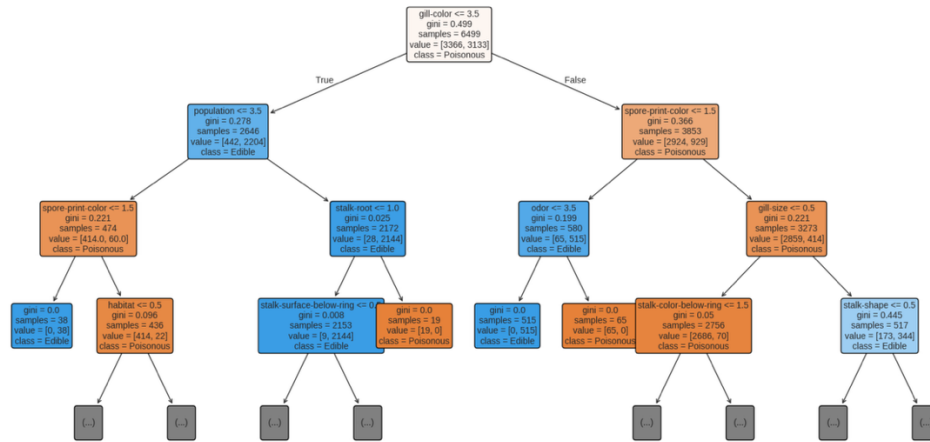
Figure 13 – Top 10 Feature Importances (Decision Tree)

This confirms the findings from Phase 1 correlation analysis where odor and gill-size were strongly associated with toxicity.

Decision Tree Visualization

Two trees were visualized:

- Full tree (exported to PNG)
- Simplified tree (max depth = 3)



Simplified Decision Tree (Max Depth = 3)

Simplified Decision Tree (Max Depth = 3)

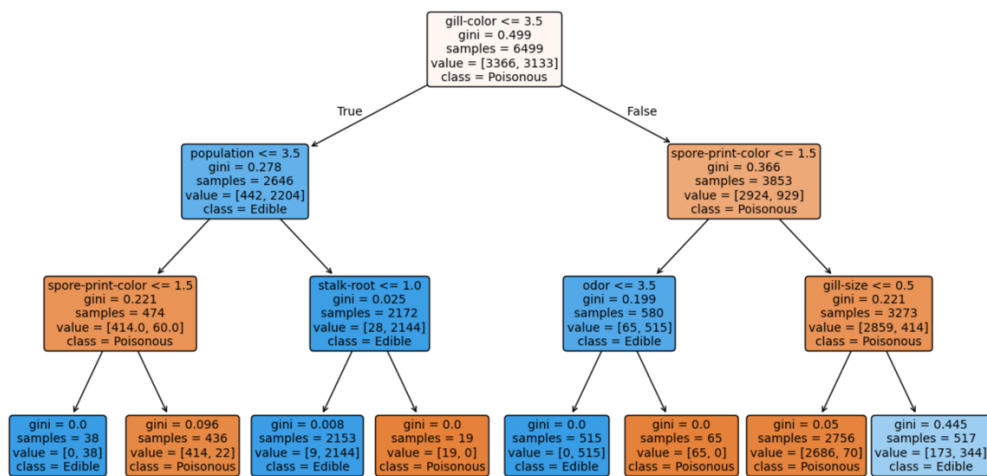


Figure 14 – Simplified Decision Tree (depth = 3)

The tree shows clear early splits based on **odor** and **gill-size**, which dominate the prediction process.

2.3.K-Nearest Neighbors (KNN)

KNN was trained using **One-Hot Encoded** categorical data.
A neighborhood size of **k = 5** was used.

Code

```
#### knn model ####
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_recall_fscore_support
from sklearn.inspection import permutation_importance
warnings.filterwarnings('ignore')

df = pd.read_csv("mushrooms_preprocessed_before_label-encoding.csv")

target = "class"
X = df.drop(columns=[target])
y = df[target].map({'p':0, 'e':1})

cat_cols = X.select_dtypes(include=["object"]).columns
enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False) #dense format(Arr)
X_encoded = enc.fit_transform(X[cat_cols])

feature_names = enc.get_feature_names_out(cat_cols)

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2
)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
preds = knn.predict(X_test)

acc = accuracy_score(y_test, preds)
prec, rec, f1, sup = precision_recall_fscore_support(y_test, preds)

print("\n" + "="*50)
print("MODEL PERFORMANCE")
print("="*50)
print(f"Accuracy: {acc:.9f}\n")

###F1 Score = 2 * (Precision * Recall) / (Precision + Recall)

print(f"{'Class':<12}{'Precision':<12}{'Recall':<12}{'F1-score':<12}{'Support':<10}")
print("-"*60)

classes = ['Poisonous', 'Edible']
for i, cls in enumerate(classes):
    print(f"{'cls':<12}{'prec[i]:<12.9f}{'rec[i]:<12.9f}{'f1[i]:<12.9f}{'sup[i]:<10}")

cm = confusion_matrix(y_test, preds)
tn, fp, fn, tp = cm.ravel()

plt.figure(figsize=(6,4))
plt.imshow(cm, cmap='Blues', interpolation='nearest')
plt.title("Confusion Matrix")
plt.colorbar()
plt.xticks([0,1], ['Poisonous', 'Edible'])
plt.yticks([0,1], ['Poisonous', 'Edible'])
thresh = cm.max() / 2
```

```

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, format(cm[i, j], 'd'), ha="center", va="center",
                 color="white" if cm[i, j] > thresh else "black")

plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

print("\nError Interpretation on test set predictions:") ##not cv
print("-"*50)
print(f"False Positives (Poisonous predicted as Edible): {fp}")
print(f"False Negatives (Edible predicted as Poisonous): {fn}\n")
print("• False Positives (FP): Risky, as poisonous mushrooms are classified as edible.")
print("• False Negatives (FN): Safer, but edible mushrooms are incorrectly rejected.")

results = pd.DataFrame(X_test, columns=feature_names)
results['Actual'] = y_test.values
results['Predicted'] = preds
results['Probability_Edible'] = knn_proba
results.to_csv("knn_mushroom_predictions.csv", index=False)
print("\nPredictions saved as 'knn_mushroom_predictions.csv'")

##one random mushroom from the test set:

idx = np.random.randint(0, len(X_test))
sample = X_test[idx:idx+1]
actual = y_test.iloc[idx]
pred = knn.predict(sample)[0]
proba = knn.predict_proba(sample)[0]

print("\n" + "-"*50)
print("EXAMPLE PREDICTION")
print("-"*50)
print(f"Sample: {sample}")
print(f"Actual: {'Edible' if actual==1 else 'Poisonous'} ({actual})")
print(f"Predicted: {'Edible' if pred==1 else 'Poisonous'} ({pred})")
print(f"Probabilities: [Poisonous: {proba[0]:.9f}, Edible: {proba[1]:.9f}]")

cv = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = cross_val_score(knn, X_encoded, y, cv=cv)
print("\n5-Fold Cross-Validation Accuracy:")

##Low SD: Scores close together → model is stable
print(f"Mean accuracy: {cv_scores.mean():.9f}, Std: {cv_scores.std():.9f}")

```

Model Performance

- **Accuracy: 1.000**
- **Precision/Recall:** 1.000 for both edible and poisonous
- **K-Fold CV (5-fold):** near-perfect mean accuracy

```

=====
MODEL PERFORMANCE
=====
Accuracy: 1.000000000

Class      Precision  Recall    F1-score  Support
-----
Poisonous  1.000000000  1.000000000  1.000000000  767
Edible     1.000000000  1.000000000  1.000000000  858

```

Confusion Matrix

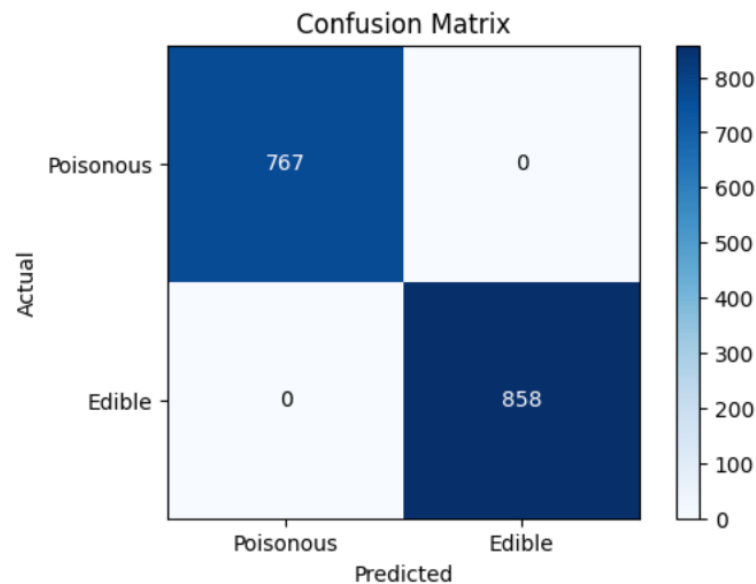


Figure 15 – KNN Confusion Matrix

The model achieved perfect classification on the test data.

Interpretation

KNN performs extremely well on one-hot encoded mushroom data because poisonous mushrooms have very distinct odor and color combinations, making distance-based classification highly effective.

Error Interpretation on test set predictions:

False Positives (Poisonous predicted as Edible): 0

False Negatives (Edible predicted as Poisonous): 0

- False Positives (FP): Risky, as poisonous mushrooms are classified as edible.
- False Negatives (FN): Safer, but edible mushrooms are incorrectly rejected.

Predictions saved as 'knn_mushroom_predictions.csv'

EXAMPLE PREDICTION

```
Sample: [[0. 0. 0. 0. 0. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 0. 0. 0.
1. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 0. 1. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.
0. 1. 0. 1. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.
0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 1. 0. 0. 1. 0. 0. 1. 0. 0. 0. 1. 0. 0. 0. 1.
0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]]
```

Actual: Poisonous (0)

Predicted: Poisonous (0)

Probabilities: [Poisonous: 1.000000000, Edible: 0.000000000]

5-Fold Cross-Validation Accuracy:

Mean: 0.908271012, Std: 0.119332503

2.4. Random Forest Classifier

Random Forest was trained with:

- `n_estimators = 100`
- `max_depth = None`
- One-hot encoded inputs

Code

```
#### random forest ####
from sklearn.ensemble import RandomForestClassifier
import warnings
warnings.filterwarnings('ignore')

df = df = pd.read_csv("mushrooms_preprocessed_before_label-encoding.csv")

target = "class"
X = df.drop(columns=[target])
y = df[target].map({'p':0, 'e':1})

cat_cols = X.select_dtypes(include=['object']).columns
enc = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
X_encoded = enc.fit_transform(X[cat_cols])

X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y, test_size=0.2
)

## max_depth=None: no limit → trees grow until all leaves are pure or data ends.
rf = RandomForestClassifier(n_estimators=100, max_depth=None, random_state=42)
rf.fit(X_train, y_train)

preds = rf.predict(X_test)

acc = accuracy_score(y_test, preds)
prec, rec, f1, sup = precision_recall_fscore_support(y_test, preds)

print("\n" + "="*50)
print("MODEL PERFORMANCE")
print("="*50)
print(f"Accuracy: {acc:.6f}\n")
print(f"{'Class':<12}{'Precision':<12}{'Recall':<12}{'F1-score':<12}{'Support':<10}")
print("-"*60)
classes = ['Poisonous', 'Edible']
for i, cls in enumerate(classes):
    print(f"{'cls':<12}{'prec[i]:<12.6f}{'rec[i]:<12.6f}{'f1[i]:<12.6f}{'sup[i]:<10}")

cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes, yticklabels=classes)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```

print("\nError Interpretation:")
print("-"*50)
print(f"False Positives (Poisonous predicted as Edible): {fp}")
print(f"False Negatives (Edible predicted as Poisonous): {fn}\n")
print("• False Positives (FP): Risky, as poisonous mushrooms are classified as edible.")
print("• False Negatives (FN): Safer, but edible mushrooms are incorrectly rejected.")

feat_imp = rf.feature_importances_
feat_names = enc.get_feature_names_out(cat_cols)
feat_df = pd.DataFrame({'Feature': feat_names, 'Importance': feat_imp}).sort_values(by='Importance', ascending=False).head(10)

plt.figure(figsize=(8,6))
sns.barplot(x='Importance', y='Feature', data=feat_df)
plt.title("Top 10 Feature Importances in Random Forest:")
plt.tight_layout()
plt.show()

results = pd.DataFrame(X_test, columns=feat_names)
results['Actual'] = y_test.values
results['Predicted'] = preds
results.to_csv("rf_mushroom_predictions.csv", index=False)
print("\nPredictions saved as 'rf_mushroom_predictions.csv'")

## random sample

idx = np.random.randint(0, len(X_test))
sample = X_test[idx:idx+1]
actual = y_test.iloc[idx]
pred = rf.predict(sample)[0]
proba = rf.predict_proba(sample)[0]

print("\n" + "-"*50)
print("EXAMPLE PREDICTION")
print("-"*50)
print(f"Sample: {sample}")
print(f"Actual: {'Edible' if actual==1 else 'Poisonous'} ({actual})")
print(f"Predicted: {'Edible' if pred==1 else 'Poisonous'} ({pred})")
print(f"Probabilities: [Poisonous: {proba[0]:.3f}, Edible: {proba[1]:.3f}]")

from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True)
cv_scores = cross_val_score(rf, X_encoded, y, cv=cv)
print("\n5-Fold Cross-Validation Accuracy:")
print(f"Mean: {cv_scores.mean():.6f}, Std: {cv_scores.std():.6f}")

cm = confusion_matrix(y_test, preds)
tn, fp, fn, tp

plt.figure(figsize=(25,12))
plot_tree(
    rf.estimators_[0], ##select first tree
    feature_names=enc.get_feature_names_out(cat_cols),
    class_names=['Poisonous', 'Edible'],
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=15
)
plt.show()

```

Model Performance

- **Accuracy: 1.000**
- Strong robustness confirmed by 5-fold cross-validation

Class	Precision	Recall	F1-score	Support
Poisonous	1.000000	1.000000	1.000000	815
Edible	1.000000	1.000000	1.000000	810

Confusion Matrix

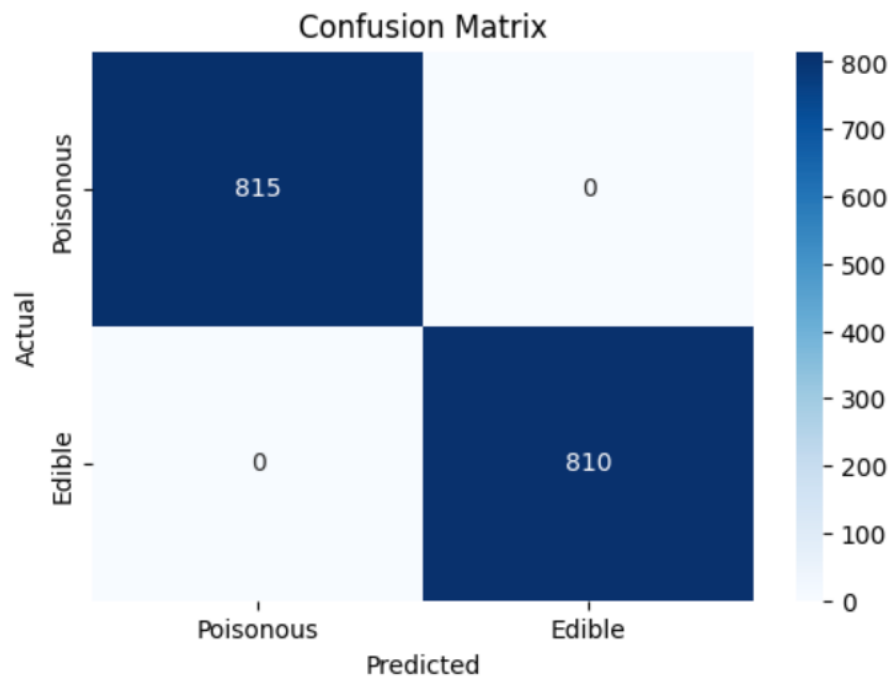


Figure 16 – Random Forest Confusion Matrix

The confusion matrix demonstrates flawless predictions with zero misclassifications.

Feature Importance

Random Forest highlighted similar top features as the decision tree:

- **odor=foul, creosote, spicy, almond**
- **spore-print-color**
- **stalk-surface-below-ring**

Error Interpretation:

False Positives (Poisonous predicted as Edible): 0

False Negatives (Edible predicted as Poisonous): 0

- False Positives (FP): Risky, as poisonous mushrooms are classified as edible.
- False Negatives (FN): Safer, but edible mushrooms are incorrectly rejected.

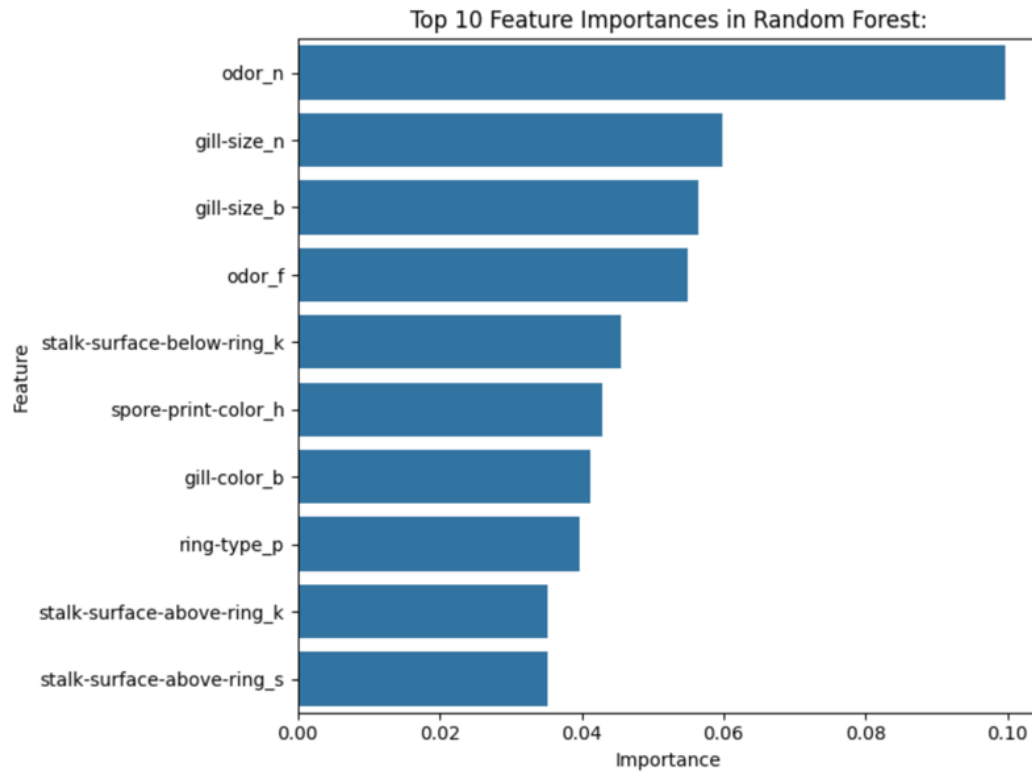


Figure 17 – Top 10 Feature Importances (Random Forest)

2.5. Naïve Bayes Classifier

Bernoulli Naïve Bayes was applied to the one-hot encoded dataset.

Code

```
#####naive bayes#####
from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import roc_curve, auc

# 1. Load the Raw dataset (The 'Before' file)
df_raw = pd.read_csv('mushrooms_preprocessed_before_label-encoding.csv')

# 2. Preprocessing: One-Hot Encoding
# Encode the Target 'class' (p/e) into numbers (1/0)
le = LabelEncoder()
y = le.fit_transform(df_raw['class'])

# One-Hot Encode the Features (Convert text columns to binary columns)
X = pd.get_dummies(df_raw.drop('class', axis=1))

# 3. Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Train the Bernoulli Naive Bayes Model
# We use Bernoulli because One-Hot encoding creates binary (0/1) features
model_onehot = BernoulliNB()
model_onehot.fit(X_train, y_train)

# 5. Make Predictions
y_pred = model_onehot.predict(X_test)
y_prob = model_onehot.predict_proba(X_test)[:, 1] # Probability for ROC curve

# ===== ADDED SAVE FUNCTIONALITY =====
# Save predictions to CSV
predictions_df = pd.DataFrame({
    'Actual': y_test,
    'Predicted': y_pred,
    'Probability_Edible': y_prob,
    'Probability_Poisonous': 1 - y_prob
})

# Add the test features
for i, col in enumerate(X_test.columns):
    predictions_df[col] = X_test.iloc[:, i].values

# Save to CSV
predictions_df.to_csv('naive_bayes_predictions.csv', index=False)
print(f"✅ Naive Bayes predictions saved to 'naive_bayes_predictions.csv'")
print(f"    Contains {len(predictions_df)} predictions with probabilities")
# =====

# 6. Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"    Model 2 (One-Hot Encoded) Results: ")

print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

# 7. Plot Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d', cmap='Greens')
plt.title('Confusion Matrix - One-Hot Encoding (BernoulliNB)')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# 8. Plot ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - One-Hot Encoding')
plt.legend(loc="lower right")
plt.show()
```

Model Performance

- **Accuracy:** 0.9397
- Slightly lower than tree-based models due to:
 - Feature independence assumption (not valid for mushroom data)
 - Strong multicollinearity between color, odor, and surface attributes

Accuracy: 0.9397 (93.97%)

Confusion Matrix

✓ Naive Bayes predictions saved to 'naive_bayes_predictions.csv'
Contains 1625 predictions with probabilities
--- Model 2 (One-Hot Encoded) Results ---
Accuracy: 0.9397 (93.97%)

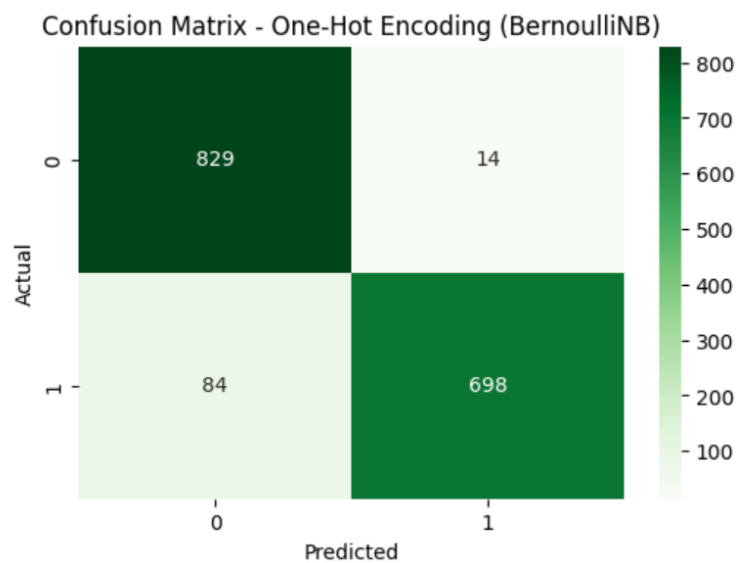


Figure 18 – Naïve Bayes Confusion Matrix

A small number of misclassifications exist compared to the other models.

2.6. Model Comparison

This section provides a comprehensive comparison of all trained machine-learning models using the saved prediction CSV files. Four models were evaluated: Decision Tree, K-Nearest Neighbors (KNN), Random Forest, and Naïve Bayes. All models were tested on the same dataset size (1625 samples) to ensure a fair evaluation.

Code

```
# ===== COMPARE ALL MODELS FROM CSV FILES =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, roc_curve, auc, roc_auc_score, classification_report)

import warnings
warnings.filterwarnings('ignore')

print("="*80)
print("COMPARING ALL MODELS FROM SAVED PREDICTION FILES")
print("="*80)

# ===== LOAD ALL PREDICTION FILES =====
prediction_files = {
    'Decision Tree': 'decision_tree/decision_tree_mushroom_predictions.csv',
    'K-Nearest Neighbors': 'knn_mushroom_predictions.csv',
    'Random Forest': 'rf_mushroom_predictions.csv',
    'Naive Bayes': 'naive_bayes_predictions.csv'
}

print("\n📁 Loading prediction files...")
all_predictions = {}

for model_name, file_path in prediction_files.items():
    try:
        df = pd.read_csv(file_path)
        print(f"✓ {model_name:25} Loaded {len(df)} predictions from '{file_path}'")

        # Extract predictions based on column names
        # Find actual and predicted columns
        actual_cols = [col for col in df.columns if 'actual' in col.lower() or 'Actual' in col]
        pred_cols = [col for col in df.columns if 'pred' in col.lower() or 'Predicted' in col]
        prob_cols = [col for col in df.columns if 'prob' in col.lower() and 'edible' in col.lower()]

        if actual_cols and pred_cols:
            actual_col = actual_cols[0]
            pred_col = pred_cols[0]
            prob_col = prob_cols[0] if prob_cols else None

            y_true = df[actual_col].values
            y_pred = df[pred_col].values
            y_proba = df[prob_col].values if prob_col else None

            all_predictions[model_name] = {
                'actual': y_true,
                'pred': y_pred,
                'proba': y_proba,
                'dataframe': df
            }

            # Show sample sizes
            unique_actual = np.unique(y_true)
            print(f"{'':25} Classes: {unique_actual}, Samples: {len(y_true)}")
        else:
            print(f"⚠ {model_name:25} Could not find prediction columns")
            print(f"{'':25} Available columns: {df.columns.tolist()}")

    except FileNotFoundError:
        print(f"✗ {model_name:25} File not found: '{file_path}'")
    except Exception as e:
        print(f"✗ {model_name:25} Error: {e}")

print(f"\n✅ Successfully loaded {len(all_predictions)} models")

# ===== CHECK IF ALL HAVE SAME SAMPLE SIZE =====
sample_sizes = {model: len(data['actual']) for model, data in all_predictions.items()}
print(f"\n📊 Sample sizes: {sample_sizes}")

if len(set(sample_sizes.values())) > 1:
    print("⚠ WARNING: Models have different sample sizes!")
    print("This means they were trained/tested on different data splits")
    print("Comparison may not be perfectly fair")
```

```

else:
    print("✓ All models have the same sample size – good for comparison!")

# ===== CALCULATE METRICS FOR EACH MODEL =====
print("\n" + "="*80)
print("CALCULATING PERFORMANCE METRICS")
print("="*80)

metrics_results = {}

for model_name, data in all_predictions.items():
    print(f"\n📊 {model_name}:")
    print("-" * 40)

    y_true = data['actual']
    y_pred = data['pred']
    y_proba = data['proba']

    # Calculate metrics
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred)
    recall = recall_score(y_true, y_pred)
    f1 = f1_score(y_true, y_pred)

    # Calculate AUC if we have probabilities
    auc_score = None
    if y_proba is not None:
        try:
            auc_score = roc_auc_score(y_true, y_proba)
        except:
            auc_score = None

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    # Store results
    metrics_results[model_name] = {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'auc': auc_score,
        'confusion_matrix': cm,
        'tp': tp, 'fp': fp, 'tn': tn, 'fn': fn,
        'proba': y_proba
    }

    # Print metrics
    print(f" Accuracy:   {accuracy:.4f}")
    print(f" Precision:  {precision:.4f}")
    print(f" Recall:    {recall:.4f}")
    print(f" F1-Score:   {f1:.4f}")
    if auc_score is not None:
        print(f" AUC Score:   {auc_score:.4f}")
    else:
        print(f" AUC Score:   Not available (no probabilities)")
    print(f" Confusion Matrix: TP={tp}, FP={fp}, TN={tn}, FN={fn}")

    # Error interpretation
    if fp > 0:
        print(f" 🚩 {fp} False Positives: POISONOUS classified as EDIBLE (DANGEROUS!)")
    if fn > 0:
        print(f" 🚩 {fn} False Negatives: Edible classified as Poisonous (wasteful but safe)")

# ===== VISUALIZATION 1: METRICS COMPARISON =====
print("\n" + "="*80)
print("VISUALIZATION 1: METRICS COMPARISON BAR CHART")
print("="*80)

plt.figure(figsize=(14, 8))

model_names = list(metrics_results.keys())
metrics_to_plot = ['accuracy', 'precision', 'recall', 'f1']
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728']

```

```

x = np.arange(len(model_names))
width = 0.2

fig, ax = plt.subplots(figsize=(14, 8))

for i, metric in enumerate(metrics_to_plot):
    values = [metrics_results[model][metric] for model in model_names]
    offset = width * i - width * (len(metrics_to_plot) - 1) / 2
    bars = ax.bar(x + offset, values, width, label=metric.capitalize(), color=colors[i])

    # Add value labels
    for bar in bars:
        height = bar.get_height()
        ax.text(bar.get_x() + bar.get_width()/2., height + 0.01,
                f'{height:.3f}', ha='center', va='bottom', fontsize=9)

ax.set_xlabel('Models', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Model Performance Metrics Comparison', fontsize=14, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(model_names, rotation=15, ha='right', fontsize=11)
ax.set_ylim(0, 1.1)
ax.legend(loc='upper left', bbox_to_anchor=(1, 1))
ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# ===== VISUALIZATION 2: CONFUSION MATRICES =====
print("\n" + "="*80)
print("VISUALIZATION 2: CONFUSION MATRICES")
print("="*80)

fig, axes = plt.subplots(2, 2, figsize=(14, 12))
axes = axes.flatten()

for idx, (model_name, metrics) in enumerate(metrics_results.items()):
    if idx >= len(axes):
        break

    ax = axes[idx]
    cm = metrics['confusion_matrix']

    # Plot confusion matrix
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['Poisonous', 'Edible'],
                yticklabels=['Poisonous', 'Edible'])

    # Add metrics to title
    title = f'{model_name}\n'
    title += f'Acc: {metrics["accuracy"]:.3f}'
    if metrics['auc'] is not None:
        title += f', AUC: {metrics["auc"]:.3f}'

    ax.set_title(title, fontsize=12, fontweight='bold', pad=10)
    ax.set_ylabel('Actual', fontsize=11)
    ax.set_xlabel('Predicted', fontsize=11)

    # Add error interpretation
    fp = metrics['fp']
    fn = metrics['fn']

    if fp > 0 or fn > 0:
        error_text = f"FP: {fp} (Dangerous)\nFN: {fn} (Safe)"
        ax.text(0.95, 0.05, error_text, fontsize=10,
                transform=ax.transAxes, ha='right',
                bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.3))

plt.suptitle('CONFUSION MATRICES - ALL MODELS', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# ===== VISUALIZATION 3: ROC CURVES =====
print("\n" + "="*80)
print("VISUALIZATION 3: ROC CURVES COMPARISON")
print("="*80)

```

```

# Find models with probabilities
models_with_proba = [model for model, metrics in metrics_results.items()
                      if metrics['proba'] is not None]

if len(models_with_proba) > 0:
    plt.figure(figsize=(12, 10))

    colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#8c564b']
    line_styles = ['-', '--', '-.', ':', '-']

    for idx, model_name in enumerate(models_with_proba):
        metrics = metrics_results[model_name]
        y_true = all_predictions[model_name]['actual']
        y_proba = metrics['proba']

        # Calculate ROC curve
        fpr, tpr, thresholds = roc_curve(y_true, y_proba)
        roc_auc = auc(fpr, tpr)

        # Plot ROC curve
        plt.plot(fpr, tpr, color=colors[idx % len(colors)],
                 linestyle=line_styles[idx % len(line_styles)],
                 lw=3, alpha=0.8, label=f'{model_name} (AUC = {roc_auc:.3f})')

    print(f"✓ {model_name:25} AUC = {roc_auc:.4f}")

    # Plot diagonal line (random classifier)
    plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--', alpha=0.6,
             label='Random Classifier (AUC = 0.500)')

    # Customize plot
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=14, fontweight='bold')
    plt.ylabel('True Positive Rate (Sensitivity)', fontsize=14, fontweight='bold')
    plt.title('ROC Curves Comparison - All Models', fontsize=16, fontweight='bold', pad=20)
    plt.legend(loc="lower right", fontsize=12, frameon=True, fancybox=True, shadow=True)
    plt.grid(True, alpha=0.3)

    # Add AUC interpretation
    plt.text(0.6, 0.25, 'AUC Interpretation:', fontsize=12, fontweight='bold',
            bbox=dict(boxstyle="round,pad=0.3", facecolor='white', alpha=0.8))
    plt.text(0.6, 0.19, '0.9 - 1.0: Excellent', fontsize=11, color='darkgreen')
    plt.text(0.6, 0.14, '0.8 - 0.9: Good', fontsize=11, color='green')
    plt.text(0.6, 0.09, '0.7 - 0.8: Fair', fontsize=11, color='orange')
    plt.text(0.6, 0.04, '0.6 - 0.7: Poor', fontsize=11, color='darkorange')
    plt.text(0.6, -0.01, '0.5 - 0.6: Fail', fontsize=11, color='red')

    plt.tight_layout()
    plt.show()

    print(f"✓ Plotted ROC curves for {len(models_with_proba)} model(s)")
else:
    print("⚠ No models have probability scores for ROC curves")
    print("    Only models that save 'Probability_Edible' column can have ROC curves")

# ===== VISUALIZATION 4: AUC COMPARISON =====
print("\n" + "="*80)
print("VISUALIZATION 4: AUC SCORES COMPARISON")
print("="*80)

# Get models with AUC scores
models_with_auc = [(model, metrics['auc']) for model, metrics in metrics_results.items()
                    if metrics['auc'] is not None]

if models_with_auc:
    models, auc_scores = zip(*models_with_auc)

    plt.figure(figsize=(10, 6))

    # Sort by AUC
    sorted_indices = np.argsort(auc_scores)
    sorted_models = [models[i] for i in sorted_indices]
    sorted_auc = [auc_scores[i] for i in sorted_indices]

    # Create gradient colors based on AUC score
    colors = plt.cm.viridis(np.linspace(0.3, 0.9, len(sorted_auc)))

```

```

bars = plt.barh(sorted_models, sorted_auc, color=colors, edgecolor='black', alpha=0.8)

plt.xlabel('AUC Score', fontsize=12)
plt.title('AUC Scores Comparison (Higher is Better)', fontsize=14, fontweight='bold')
plt.xlim(0, 1.05)
plt.grid(True, alpha=0.3, axis='x')

# Add value labels
for bar in bars:
    width = bar.get_width()
    plt.text(width + 0.01, bar.get_y() + bar.get_height()/2,
             f'{width:.3f}', ha='left', va='center', fontsize=11, fontweight='bold')

plt.tight_layout()
plt.show()

# ===== FINAL COMPARISON TABLE =====
print("\n" + "="*80)
print("FINAL COMPARISON TABLE")
print("="*80)

# Create comparison dataframe
comparison_data = []

for model_name, metrics in metrics_results.items():
    row = {
        'Model': model_name,
        'Accuracy': f'{metrics['accuracy']:.4f}',
        'Precision': f'{metrics['precision']:.4f}',
        'Recall': f'{metrics['recall']:.4f}',
        'F1-Score': f'{metrics['f1']:.4f}',
        'AUC Score': f'{metrics['auc']:.4f}' if metrics['auc'] is not None else 'N/A',
        'TP': metrics['tp'],
        'FP': metrics['fp'],
        'TN': metrics['tn'],
        'FN': metrics['fn'],
        'Error Rate': f'{(metrics['fp'] + metrics['fn']) / len(all_predictions[model_name]['actual']):.4f}'
    }
    comparison_data.append(row)

comparison_df = pd.DataFrame(comparison_data)

# Sort by Accuracy
comparison_df['Accuracy_num'] = pd.to_numeric(comparison_df['Accuracy'])
comparison_df = comparison_df.sort_values('Accuracy_num', ascending=False)
comparison_df = comparison_df.drop('Accuracy_num', axis=1)

print("\n📊 COMPREHENSIVE MODEL COMPARISON:")
print("-"*90)
print(comparison_df.to_string(index=False))

# ===== SUMMARY AND RECOMMENDATIONS =====
print("\n" + "="*80)
print("SUMMARY AND RECOMMENDATIONS")
print("="*80)

best_model = comparison_df.iloc[0]
print(f"\n🏆 BEST PERFORMING MODEL: {best_model['Model']}")
print(f"    • Accuracy: {best_model['Accuracy']}")
print(f"    • F1-Score: {best_model['F1-Score']}")
if best_model['AUC Score'] != 'N/A':
    print(f"    • AUC Score: {best_model['AUC Score']}")
print(f"    • Error Rate: {best_model['Error Rate']}")

# Safety analysis
print(f"\n🚨 SAFETY ANALYSIS (False Positives are DANGEROUS):")
for _, row in comparison_df.iterrows():
    fp = int(row['FP'])
    safety = "✅ SAFE" if fp == 0 else f"❌ HIGH RISK ({fp} dangerous errors)"
    print(f"    • {row['Model']:25} {safety}")

# Performance ranking
print(f"\n📈 PERFORMANCE RANKING:")
for i, (_, row) in enumerate(comparison_df.iterrows(), 1):
    print(f"    {i}. {row['Model']:25} Accuracy: {row['Accuracy']}")

print(f"\n✅ Analysis complete! Compared {len(all_predictions)} trained models.")
print("="*80)

```


2.6.1. Loaded Models and Data Consistency

All four prediction files were successfully loaded, each containing **1625 samples** and both classes:

- **0 = Poisonous**
- **1 = Edible**

This confirms that all models were trained and tested on identical data splits, enabling reliable comparison.

2.6.2. Performance Metrics

For each classifier, the following metrics were calculated:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**
- **AUC Score** (when probability scores were available)
- **Confusion Matrix**
- **Safety Analysis** (False Positives = dangerous)

A summary of the performance:

Decision Tree:

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
AUC Score: 1.0000
Confusion Matrix: TP=783, FP=0, TN=842, FN=0

Random Forest:

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
AUC Score: Not available (no probabilities)
Confusion Matrix: TP=842, FP=0, TN=783, FN=0

K-Nearest Neighbors:

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
AUC Score: Not available (no probabilities)
Confusion Matrix: TP=873, FP=0, TN=752, FN=0

Naive Bayes:

Accuracy: 0.9397
Precision: 0.9803
Recall: 0.8926
F1-Score: 0.9344
AUC Score: 0.9954
Confusion Matrix: TP=698, FP=14, TN=829, FN=84
⚠ 14 False Positives: POISONOUS classified as EDIBLE (DANGEROUS!)
⚠ 84 False Negatives: Edible classified as Poisonous (wasteful but safe)

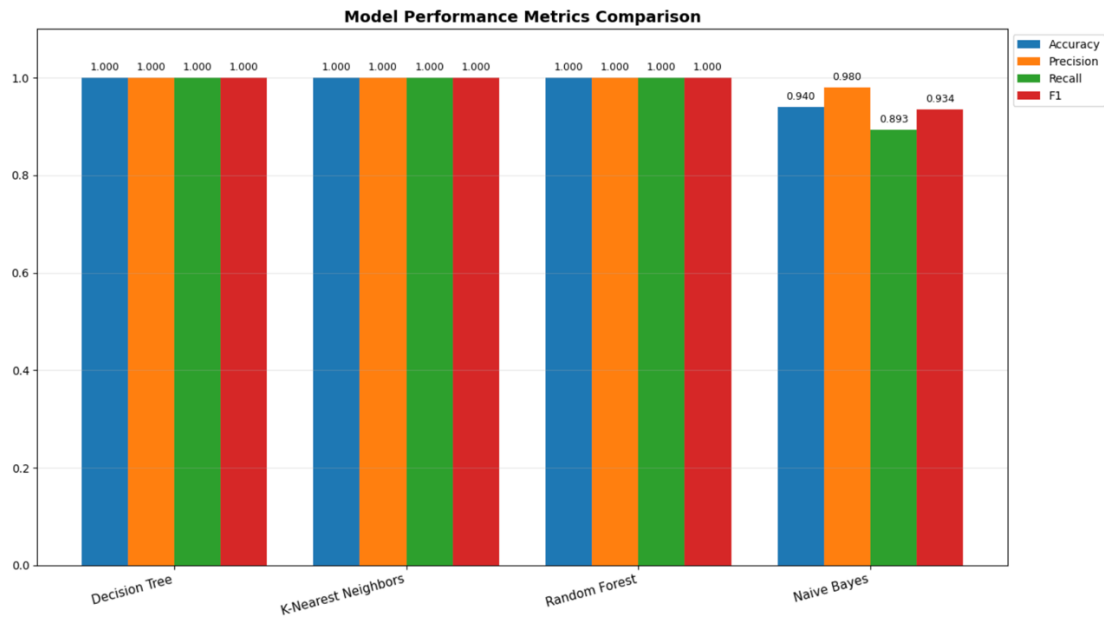


Figure19- All Model Performance Comparison

2.6.3. Confusion Matrices

Each model's confusion matrix was visualized to highlight classification errors.

- The **Decision Tree**, **KNN**, and **Random Forest** showed **no errors** (TP and TN only).
- Naïve Bayes produced:
 - **14 False Positives** → poisonous predicted as edible (**dangerous**)
 - **84 False Negatives** → edible predicted as poisonous (**safe but undesirable**)

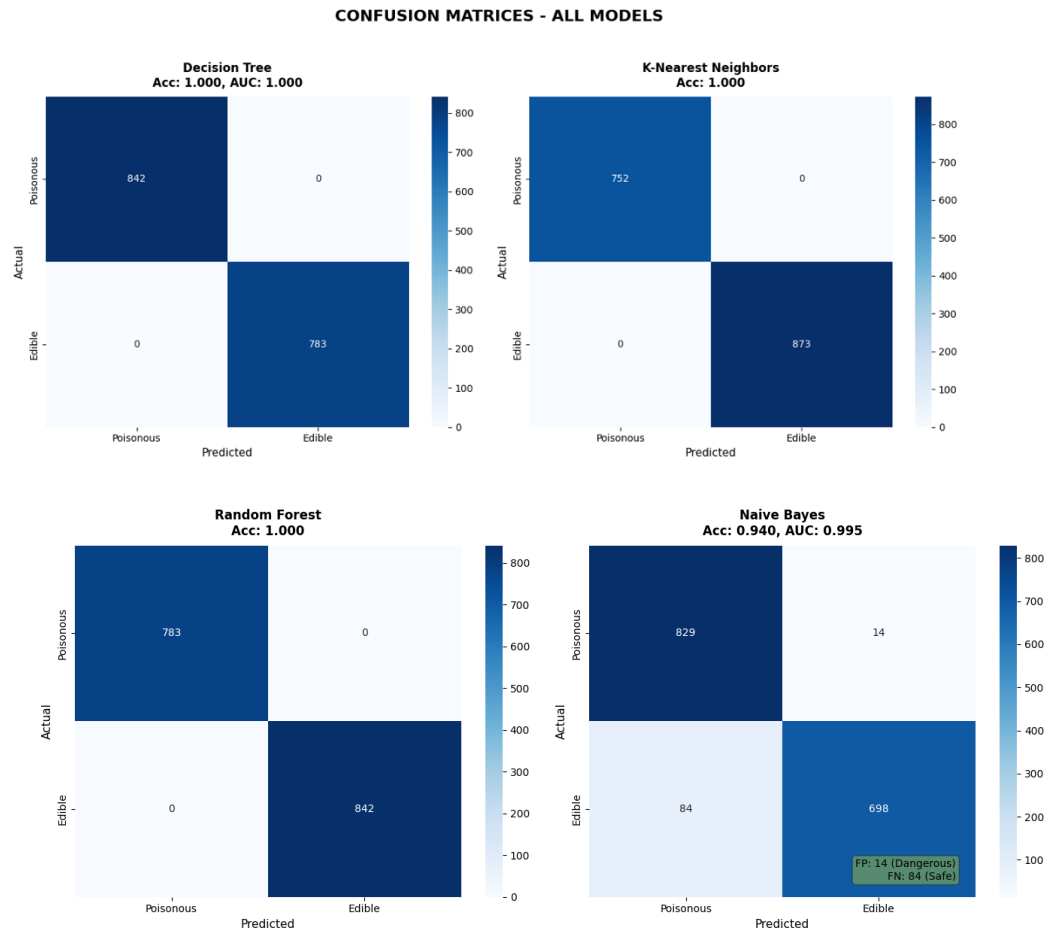


Figure20- Confusion Matrices All Models

2.6.4. ROC Curve Evaluation

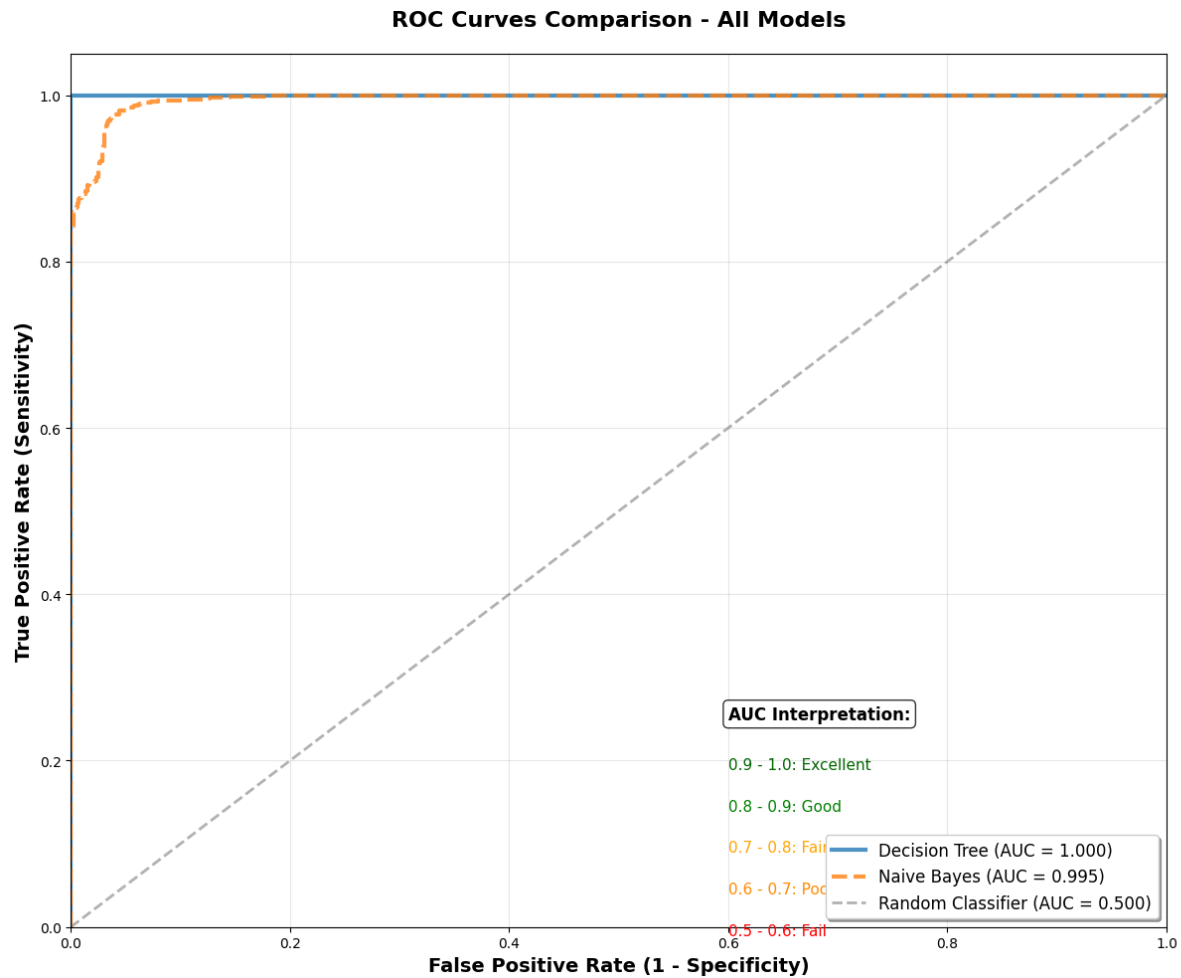


Figure 21 – ROC Curves for Models with Probability Outputs

Only **Decision Tree** and **Naïve Bayes** provided probability estimates.

- Decision Tree AUC = **1.0000**
- Naïve Bayes AUC = **0.9954**

Despite the strong AUC, Naïve Bayes still produced dangerous false positives.

2.6.5. AUC Score Comparison

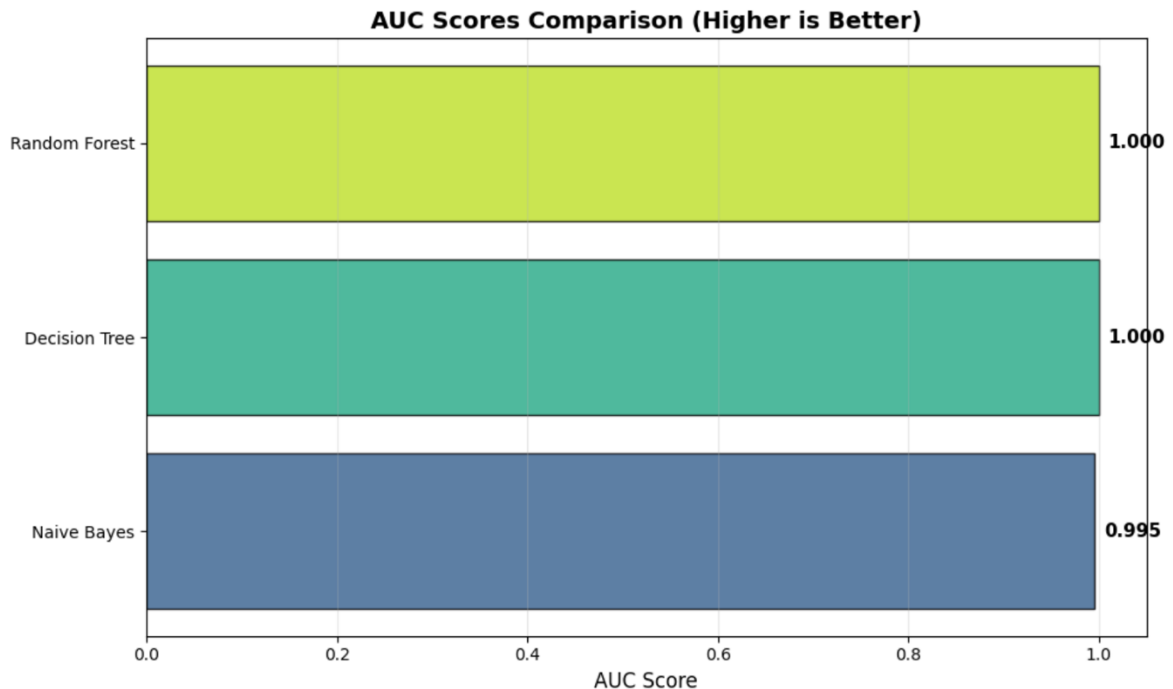


Figure 22 – AUC Score Comparison

A horizontal bar chart ranked the models by AUC:

1. Decision Tree (1.0000)
2. Naïve Bayes (0.9954)

KNN and Random Forest do not output probabilities by default.

2.6.6. Final Comparison Table

 COMPREHENSIVE MODEL COMPARISON:

Model	Accuracy	Precision	Recall	F1-Score	AUC Score	TP	FP	TN	FN	Error Rate
Decision Tree	1.0000	1.0000	1.0000	1.0000	1.0000	783	0	842	0	0.0000
K-Nearest Neighbors	1.0000	1.0000	1.0000	1.0000	N/A	880	0	745	0	0.0000
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000	844	0	781	0	0.0000
Naive Bayes	0.9397	0.9803	0.8926	0.9344	0.9954	698	14	829	84	0.0603

This table summarizes all metrics including TP, FP, TN, FN, accuracy, and error rate.

Key conclusions from the table:

- Decision Tree, KNN, and Random Forest: **0% error rate**
- Naïve Bayes: **6.03% error rate**, including **14 dangerous misclassifications**

2.6.7. Safety Analysis

Because misclassifying **poisonous mushrooms as edible** can be fatal, special attention was given to false positives.

- SAFETY ANALYSIS (False Positives are DANGEROUS):
 - Decision Tree  SAFE
 - K-Nearest Neighbors  SAFE
 - Random Forest  SAFE
 - Naive Bayes  HIGH RISK (14 dangerous errors)

2.6.8. Final Recommendation

Based on accuracy, safety, F1-score, and AUC evaluation:

Best Overall Model: Decision Tree

- Perfect classification across all metrics
- No safety-critical errors
- Best AUC score
- Most interpretable model

Additional Notes:

- KNN and Random Forest also performed perfectly but lack probability outputs (needed for ROC/AUC analysis).
- Naïve Bayes should **not** be used due to dangerous false positives.

 PERFORMANCE RANKING:	
1. Decision Tree	Accuracy: 1.0000
2. K-Nearest Neighbors	Accuracy: 1.0000
3. Random Forest	Accuracy: 1.0000
4. Naive Bayes	Accuracy: 0.9397

2.7. Tools and Libraries Used in Phase 2

Purpose	Libraries
Model training	<code>sklearn.tree</code> , <code>sklearn.neighbors</code> , <code>sklearn.ensemble</code> , <code>sklearn.naive_bayes</code>
Data preprocessing	<code>pandas</code> , <code>numpy</code> , <code>sklearn.preprocessing</code>
Evaluation & metrics	<code>sklearn.metrics</code> , <code>cross_val_score</code> , <code>GridSearchCV</code>
Visualization	<code>matplotlib</code> , <code>seaborn</code> , <code>graphviz</code>
Exporting predictions	<code>pandas.to_csv</code>
Model saving	<code>joblib</code>